

AI & IoT BASED SMART ELDERLY CARE AND EMERGENCY MONITORING SYSTEM

Higher National Diploma in Software Engineering

Final Final Documentation

2025 – 25.1P Batch



School of Computing and Engineering

National Institute of Business Management

Kandy

AI & IoT BASED SMART ELDERLY CARE AND EMERGENCY MONITORING SYSTEM

Higher National Diploma in Software Engineering

25.1P Final Project Report

Institution

School of Computing National Institute of Business Management

Program

Higher National Diploma in Software Engineering

Supervisor

Mr. Sandaruwan Herath

Batch and Year

25.1P – 2025

Submitted by:

KAHNDSE251P-004 : R. M. A. S. RATHNAYAKE

KAHNDSE251P-005 : K. P. T. N. JAYATHISSA

KAHNDSE251P-009: K. P. C. S. KAWUDUPITIYA

“The project is submitted in partial fulfillment of the requirement of the Diploma of Software Engineering at the National Institute of Business Management.”

Declaration

We hereby declare that this project does not contain any material which has been used previously without acknowledgment for a Higher National Diploma in any institute. As far as we know and believe, this project does not contain any previously published or written material by any other person or us except the due references in the text. We hereby also grant permission to our project report, in case it is accepted, to be reproduced and lent for inter-library use, and also the title and abstract of this project to be made available to external bodies.

Student Name	Student Number	Signature
RMAS RATHNAYAKE	KAHNDSE251P-004	
KPTN JAYATHISSA	KAHNDSE251P-005	
KPCS KAWUDUPITIYA	KAHNDSE251P-009	

Supervisor: Mr Sandaruwan Herath

Signature:

Date:

Abstract

Older adults living alone may be prone to falling accidents, physiological anomalies, prolonged immobility, emotional problems, or inability to seek help. The project designed and tested a combined solution consisting of an IoT wearable sensor, cloud data exchange mechanism, AI services, deterministic health-risk analysis, and caregiver app. The wearable hardware design consists of ESP32-C3 processor, MPU6050 inertial sensor, MAX30102 oximetry sensor, and manual SOS button. The cloud data layer is provided by Firebase Realtime Database used to exchange events and states between wearable devices, Python code, and mobile client. The AI layer includes Random Forest fall detection classifier based on the Smartphone Human Fall Dataset and a video-based mood analyser which is built upon RetinaFace for facial location and DeepFace emotions classifier pretrained model. To make the mood classification more reliable, temporal smoothing is applied to ensure a certain period of mood consistency, making it possible to play local suitable music upon mood recognition. Health data processing is implemented as a rules-based explanation engine producing severity, risk, reasons, and event codes.

The fall classifier was tuned with 5-fold cross-validation and evaluated on the official unseen test set. It achieved 96.91% accuracy, 94.38% fall precision, 98.69% fall recall, 96.49% fall F1-score, 99.59% ROC-AUC, and 99.43% PR-AUC, with only two missed falls among 153 fall examples. Eighteen automated software tests passed. The prototype demonstrates technically coherent end-to-end monitoring and alert orchestration. However, the fall model currently expects nine window-level engineered features from a smartphone-derived dataset; device-specific MPU6050 window extraction and prospective validation on elderly participants remain future work. Therefore, the system is positioned as a decision-support prototype rather than a certified medical device.

Keywords: elderly care; fall detection; Random Forest; IoT; Firebase; emotion recognition; health monitoring; ESP32-C3

Acknowledgement

The project team sincerely thanks ‘Mr. Sandaruwan Herath’ for his invaluable guidance, supervision, and constant support in the development of this project. We are thankful to ‘National Institute of Business Management (NIBM), Kandy Regional Centre’ for providing us with the academic environment and the resources needed to do this project.

Moreover, we are grateful to all the ‘Group 03 members’ for their dedication, collaboration, and contribution in the areas of ‘Artificial Intelligence, Internet of Things, mobile application, cloud computing, testing, and documentation’ of the project. The collaboration of each team member was essential in completing this proposed system.

Lastly, we express our gratitude to **the open-source community, software tools, libraries, and publicly available datasets** which helped us in developing and testing this academic prototype

Table of content

Contents

1.1 Introduction of the Organization and Target Context.....	12
1.3 Current Operations.....	12
1.4 Users and Responsibilities	12
1.5 Problem Definition.....	13
1.6 Project Objectives	13
1.7 Proposed Solution	14
1.8 Scope, Assumptions, and Constraints.....	14
1.9 Chapter Summary	15
2.1 Introduction.....	16
2.2 Data Collection Methods	16
2.3 Software Process Model	16
2.4 Software Development Tools	17
2.5 AI Model Development Method.....	17
2.5.2 Preprocessing and Hyper parameter Optimization	19
2.6 Testing Strategies.....	20
2.7 Implementation Plan	21
2.8 Chapter Summary	21
3.1 Introduction.....	22
3.2 Current-System Use Cases.....	22
3.2.1 Current-System Use Case Diagram	22
3.3 Proposed-System Use Cases	23
3.4 Proposed Class Model.....	24
3.5 Sequence Analysis	25
3.5.1 Fall Event	25

3.5.2 Mood Event.....	26
3.5.3 Sequence Diagrams for Each Proposed Use Case	27
3.5.4 Monitor Vital Signs.....	27
3.5.5 Detect Possible Fall.....	28
3.5.6 Request SOS	29
3.5.7 Assess Mood	29
3.5.8 Review Alert	30
3.5.9 Review History	31
3.6 Entity–Relationship and Data Analysis	31
3.7 Functional Requirements	32
3.8 Non-Functional Requirements	32
3.9 Chapter Summary	33
4.1 Introduction.....	34
4.2 Interface Design	34
I-02 — Caregiver Dashboard: Normal State	35
I-02 — Caregiver Dashboard: Critical State.....	36
I-05 — Mood Auxiliary Tracking Interface	37
I-03/I-04 — Emergency Alerts and History Interface	38
I-06 — Health Monitoring Interface.....	39
I-07 — Emergency Calling Interface.....	40
I-08 — Caregiver Profile Interface	41
I-09 — Help and Support Interface	42
I-10 — Rules and Conditions Interface	43
4.2.1 Interface Pseudocode	44
4.2.2 Interface-wise Pseudocode.....	44
4.3 Database Design.....	47
4.4 Fall Detection Design	48

4.5 Health-Risk and Alert Design.....	50
4.6 Mood Analysis Design.....	51
4.7 IoT and Cloud Integration.....	51
4.8 Report Layout Design.....	54
4.9 Security, Privacy, and Safety Controls	55
4.10 Implementation Evidence	56
4.11 Chapter Summary	57
5.1 Achievement Against Objectives.....	58
5.2 Model Evaluation and Interpretation	58
5.3 Software Verification.....	58
5.4 Strengths	59
5.5 Limitations and Threats to Validity	59
5.6 Recommended Future Work.....	60
5.7 Conclusion	61
Appendix A — Project Schedule.....	63
Appendix B — Test Summary.....	63
Appendix C — Reproduction Commands.....	63
Appendix D — Required Completion Items	64

List of Figure

Figure 1 Proposed end-to-end logical architecture	14
Figure 2 Training and test class distribution.....	18
Figure 3 Fall-model inference and alert flow	20
Figure 4 Current-System Use Case Diagram.....	22
Figure 5 Proposed-system UML use-case diagram	24
Figure 6 Core Python UML class diagram	25
Figure 7 Fall/health alert UML sequence diagram	26
Figure 8 Mood-analysis sequence.....	26
Figure 9 Sequence Diagrams for Each Proposed Use Case.....	27
Figure 10 Sequence Diagram — Monitor Vital Signs.....	28
Figure 11 Sequence Diagram — Detect Possible Fall.....	28
Figure 12 Sequence Diagram — Request SOS.....	29
Figure 13 Sequence Diagram — Assess Mood	30
Figure 14 Sequence Diagram — Review Alert	30
Figure 15 Sequence Diagram — Review History.....	31
Figure 16 Conceptual data/entity model for Firebase records	31
Figure 17 Caregiver Dashboard — Normal State.....	35
Figure 18 Caregiver Dashboard — Critical State	36
Figure 19 Mood Auxiliary Tracking Interface.....	37
Figure 20 Emergency Alerts and History Interface	38
Figure 21 Health Monitoring Interface	39
Figure 22 Emergency Calling Interface	40
Figure 23 Caregiver Profile Interface	41
Figure 24 Help and Support Interface.....	42
Figure 25 Rules and Conditions Interface	43
Figure 26 Held-out fall-model evaluation metrics.....	48
Figure 27 Held-out confusion matrix.....	49
Figure 28 Live mood-detection interface.....	51
Figure 29 Wearable sensor serial output during integration.....	52
Figure 30 Firebase Realtime Database — Latest Motion and SOS Data	52
Figure 31 Firebase Realtime Database — Environmental Sensor Data	53
Figure 32 Firebase Realtime Database — Accelerometer and Gyroscope Data	53

Figure 33 Comprehensive Care Report Interface	55
Figure 34 Automated test suite completion evidence	56
Figure 35 Fall/no-fall simulation evidence	57

List of Tables

Table 1 List of Acronyms and Abbreviations	11
Table 2 Users and Responsibilities	12
Table 3 Scope, Assumptions, and Constraints	15
Table 4 Data Collection Methods	16
Table 5 Software Development Tools	17
Table 6 Engineered Motion Features table	19
Table 7 Testing Strategies	21
Table 8 Implementation Plan	21
Table 9 Current-System Use Cases	22
Table 10 Proposed-System Use Cases	23
Table 11 Proposed Class Model	24
Table 12 Functional Requirements	32
Table 13 Non-Functional Requirements	33
Table 14 Interface Design	34
Table 15 logical relationship key	47
Table 16 Database Design	48
Table 17 metric value	50
Table 18 Health-Risk and Alert Design	50
Table 19 Report Layout Design	54
Table 20 Achievement Against Objectives	58
Table 21 Limitations and Threats to Validity	60
Table 22 Project Schedule	63
Table 23 Appendix B — Test Summary	63

List of Acronyms and Abbreviations

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
CV	Cross-Validation
ESP32-C3	Espressif Wi-Fi/Bluetooth microcontroller
F1	Harmonic mean of precision and recall
FN / FP	False Negative / False Positive
IoT	Internet of Things
MPU6050	Six-axis accelerometer and gyroscope module
PR	Precision-Recall
ROC	Receiver Operating Characteristic
SpO ₂	Peripheral oxygen saturation
UML	Unified Modeling Language

Table 1 List of Acronyms and Abbreviations

CHAPTER 1 — INTRODUCTION

1.1 Introduction of the Organization and Target Context

This is an academic engineering project made for Elderly people and their Caregivers, where it can be used effectively for some situations. The operational context of this device is a home-care or assisted living facility wherein the elderly person is being monitored by the caregiver.

1.3 Current Operations

The traditional method of home surveillance includes periodic observation and phone calls. Any kind of fall or physiological deviation may take place during the interval between those two procedures. The individual concerned might not be able to pick up the phone, and the readings from individual sensors can be deceiving as well. Emotional stability is also never considered in one low-cost monitoring process.

1.4 Users and Responsibilities

Table 2 Users and Responsibilities

User/Actor	Primary responsibility	System interaction
Elderly user	Wear the device; place finger correctly; request help	Sensor monitoring, SOS button, mood camera
Caregiver	Review status; verify alerts; respond	Mobile dashboard, notification, history
Administrator	Configure user identity and access	Firebase configuration and account setup
AI service	Infer fall/mood and explain health risk	Python processing and cloud updates
IoT device	Acquire and publish measurements	MPU6050, MAX30102, Wi-Fi, Firebase

1.5 Problem Definition

This central issue is the lack of an integrated real-time and explainable system to identify the possibility of falls and health irregularities, transmit SOS alerts, track the emotional state, and provide useful information to the caregiver at a distance. To be technically practical, such a solution needs to find a balance between sensitivity and the generation of false alarms, function under faulty network and sensor conditions, and maintain traceability.

1.6 Project Objectives

1. Create an example of wearable sensing prototype using ESP32-C3, MPU6050, MAX30102, and SOS inputs.
2. Design and test a supervised classifier for falls considering performance on fall recall and F1-scores.
3. Implement an explainable risk engine for SpO₂, heart rate, inactivity, falls, SOS, and sensor quality.
4. Implement camera-based mood analysis with temporal stabilization and optionally reacting to music.
5. Integrate the device, AI, database, and mobile interface via Firebase Realtime Database.
6. Confirm critical behavior using model metrics, simulations, and automated software tests.

1.7 Proposed Solution

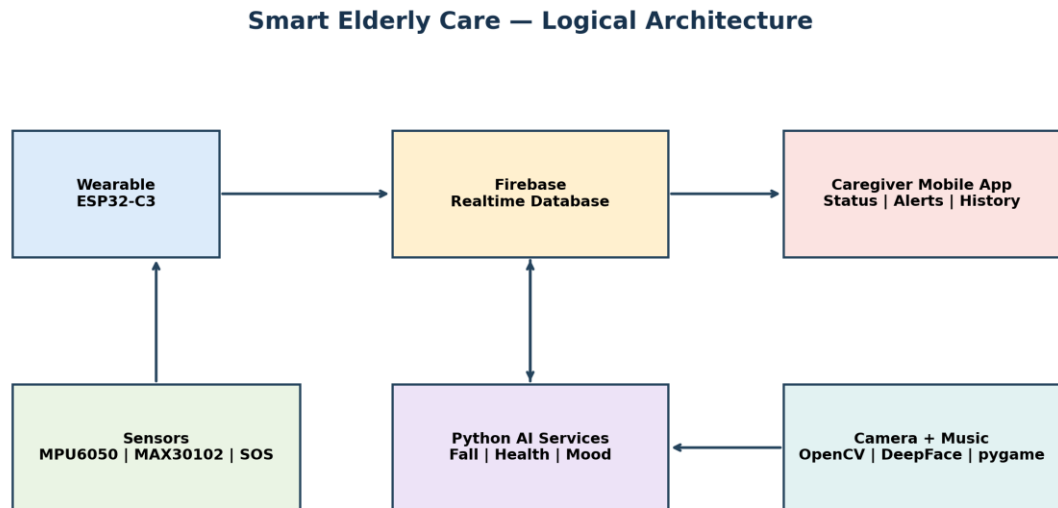


Figure 1 Proposed end-to-end logical architecture

The recommended design employs the layers concept in its implementation. The sensing will take place on the wearable device level, while the shared state and events will be persisted using Firebase. The AI and risk results will be calculated using the Python services, while the caregiver application will use the current values and alert on them.

1.8 Scope, Assumptions, and Constraints

In scope	Out of scope / constraint
Prototype sensing, cloud integration, fall inference, health rules, mood inference, alerts, mobile display	Clinical diagnosis, emergency-service dispatch, regulatory certification
Held-out evaluation on public smartphone-derived features	Prospective clinical validation on elderly users

Local music response to stable mood	Therapeutic recommendation or mental-health diagnosis
Firebase-based prototype data exchange	Guaranteed operation during prolonged Internet or power failure

Table 3 Scope, Assumptions, and Constraints

1.9 Chapter Summary

Need for operation, actors involved, objectives, architecture, and responsibility boundaries for the prototype were identified in this chapter. The next chapter elaborates on evidence collection, development of software, and performance tests.

CHAPTER 2 — METHODOLOGY

2.1 Introduction

Iteration Prototyping Methodology was chosen since it is a project with hardware, clouds, machine learning, computer vision, and mobile application. The integration risk was mitigated through the independent validation of each module before testing of scenarios.

2.2 Data Collection Methods

Evidence source	Purpose	Method
Public fall dataset	Train and test fall classification	Official Train.csv and Test.csv; labels retained, identifiers excluded
Wearable serial output	Confirm sensor acquisition and Wi-Fi start-up	ESP32-C3 serial monitor observation
Scenario simulations	Validate health and fall orchestration	Normal, warning, critical, fall, and SOS scripted inputs
Camera observations	Validate mood pipeline	Live webcam frames and sustained-label tests
Automated tests	Prevent logic regressions	Unit and integration tests across engines

Table 4 Data Collection Methods

2.3 Software Process Model

Iteration Prototyping Methodology was chosen since it is a project with hardware, clouds, machine learning, computer vision, and mobile application. The integration risk was mitigated through the independent validation of each module before testing of scenarios.

2.4 Software Development Tools

Technology	Role	Rationale
Python 3.11	AI, rules, integration, testing	Strong data-science and computer-vision ecosystem
scikit-learn	Random Forest pipeline and tuning	Reproducible preprocessing, CV, metrics, serialization
OpenCV	Camera capture and display	Real-time frame acquisition and annotation
DeepFace + RetinaFace	Emotion inference and face localization	Pretrained facial-attribute analysis with robust detector
Firebase Realtime Database	Shared live state, histories, alerts	Event-oriented synchronization between device and clients
Arduino / ESP32-C3	Embedded sensing and Wi-Fi	Compact, accessible IoT platform
pygame	Local mood-based audio playback	Simple non-blocking audio integration
unittest / pytest	Verification	Repeatable behaviour checks

Table 5 Software Development Tools

2.5 AI Model Development Method

The training and testing data files were separated from each other to ensure that the process of hyperparameter tuning would not use the test samples. There were 1,428 samples in the training dataset (614 falls and 814 non-falls), and there were 356 samples in the testing dataset (153 falls and 203 non-falls). The feature matrix consisted of all variables apart from the binary target variable “fall”, the label “label”, and the export

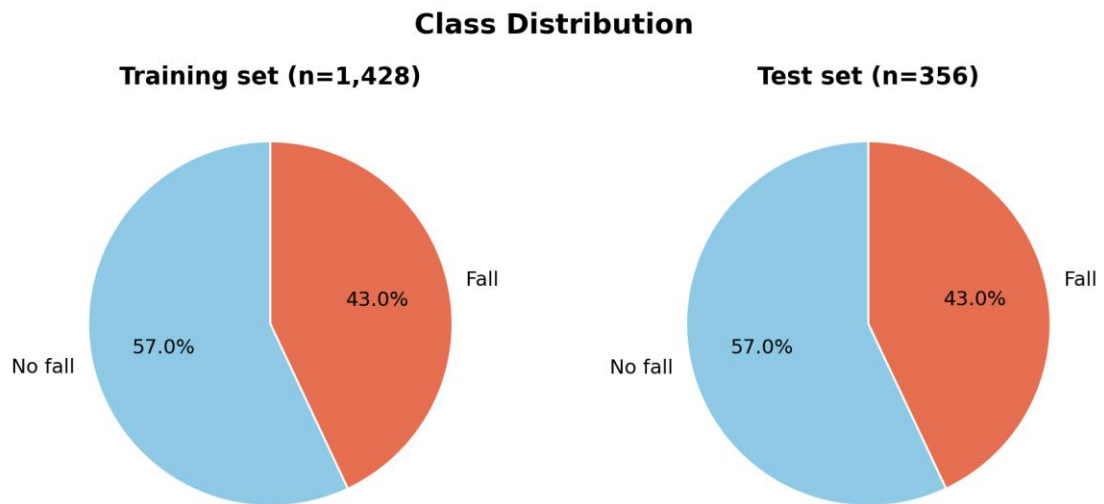


Figure 2 Training and test class distribution

Feature	Interpretation	Fall relevance
acc_max	Peak resultant acceleration in a motion window	Captures impact-like acceleration
gyro_max	Peak angular motion statistic	Captures rapid rotation
acc_kurtosis	Tail/peak concentration of acceleration	Highlights impulsive motion
gyro_kurtosis	Tail/peak concentration of angular motion	Highlights abrupt rotational events
lin_max	Peak linear-motion statistic	Represents translational intensity
acc_skewness	Acceleration distribution asymmetry	Distinguishes uneven event shapes
gyro_skewness	Angular distribution asymmetry	Describes directional rotation profile
post_gyro_max	Post-event angular statistic	Captures movement after impact

post_lin_max	Post-event linear statistic	Captures post-event recovery or stillness

Table 6 Engineered Motion Features table

2.5.2 Preprocessing and Hyper parameter Optimization

scikit-learn Pipeline with median imputation and a RandomForestClassifier was used. 30 parameter sets were randomly sampled and cross-validation was done through 5-fold cross-validation, hence 150 fitting iterations were done. Parameters included number of trees, tree depth, feature sampling, splitting criteria, minimum leaf size, and bootstrap. Optimal criterion was set to be the F1-score for class fall to balance misses and false positives. Class weights were balanced.

Fall-Detection Inference and Escalation Flow

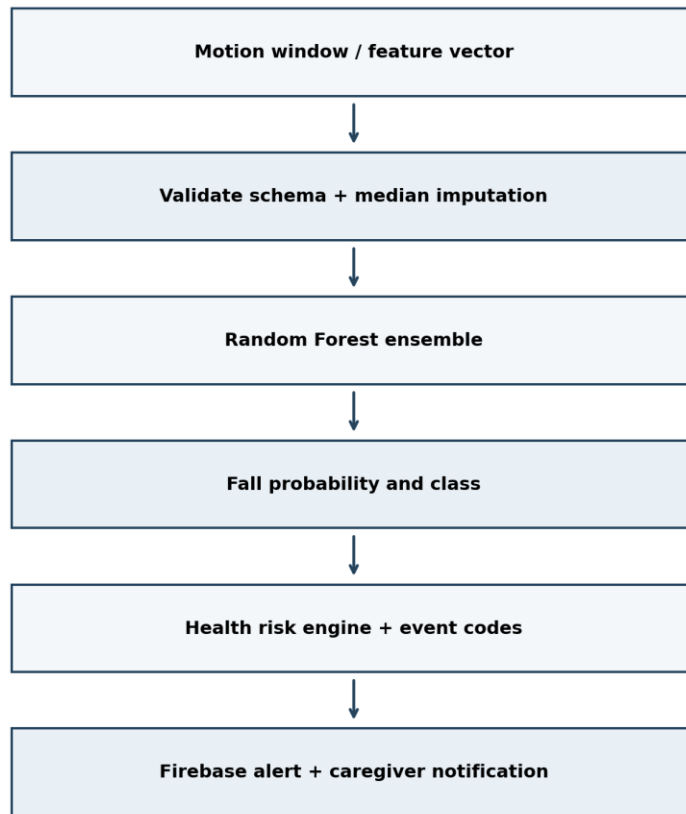


Figure 3 Fall-model inference and alert flow

2.6 Testing Strategies

Level	Examples	Acceptance focus
Unit	Health thresholds, alert mapping, smoother behaviour	Correct output for controlled inputs
Integration	System engine + model + Firebase adapter	Schema and method compatibility
Model	Official test set, confusion matrix, AUC metrics	Generalization to unseen examples
Scenario	Normal movement, recorded fall, SOS, low SpO ₂	End-to-end observable response

Usability	Readable status, severity, reason, history	Actionable caregiver interpretation
-----------	--	-------------------------------------

Table 7 Testing Strategies

2.7 Implementation Plan

Phase	Activities	Deliverable
1. Requirements	Actors, events, safety boundary	Requirements baseline
2. Data and AI	Dataset audit, pipeline, tuning, evaluation	Serialized fall model and metrics
3. IoT	Sensor wiring, sampling, Wi-Fi, cloud writes	Wearable prototype
4. Application	Mobile dashboard and alerts	Caregiver interface
5. Integration	Shared Firebase schema and adapters	End-to-end prototype
6. Verification	Tests, simulations, demo rehearsal	Evidence pack and report

Table 8 Implementation Plan

2.8 Chapter Summary

Methodology included iterative development, machine learning evaluation using holdouts, explanations by rules and software testing at various levels. Chapter 3 describes the model of functional and data relationships within the system.

CHAPTER 3 — SYSTEM ANALYSIS

3.1 Introduction

System analysis converts the requirements of the stakeholders to actors, use cases, data objects, event flows, and non-functional requirements. The diagrams used in this document are logical diagrams, and they need to be changed only if the final mobile application has other names.

3.2 Current-System Use Cases

Actor	Current action	Limitation
Elderly person	Self-report incident by telephone	May be unable to call after a fall
Caregiver	Perform scheduled checks	Incidents can occur between checks
Caregiver	Interpret verbal symptoms	No continuous sensor evidence
Family/organization	Maintain informal history	Fragmented and difficult to audit

Table 9 Current-System Use Cases

3.2.1 Current-System Use Case Diagram

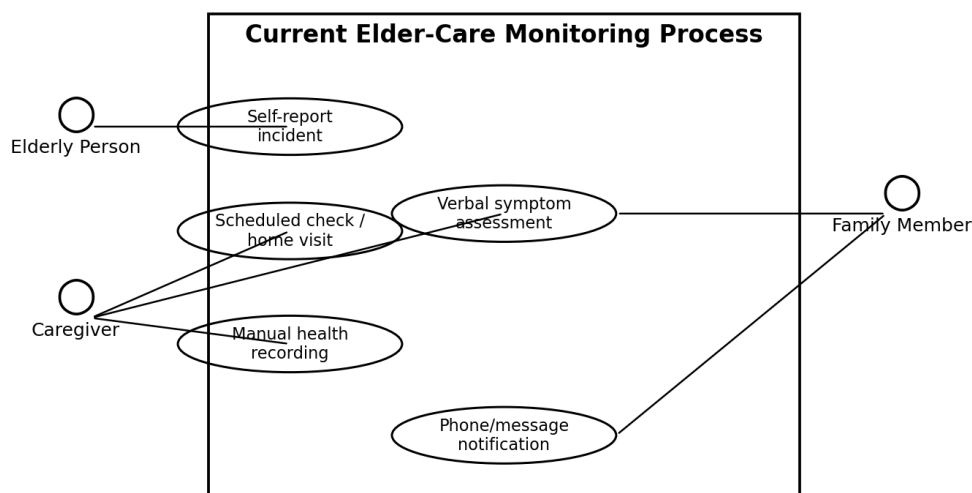


Figure 4 Current-System Use Case Diagram

The current system use case diagram shows the process of manual monitoring as discussed in the proposal. Incidents are reported manually by the elderly, and the caregivers conduct routine checking and assessment processes manually. The family is informed of the happenings via phone calls or messages.

3.3 Proposed-System Use Cases

Use case	Primary actor	Precondition	Success outcome
Monitor vital signs	Wearable	Device worn; sensor valid	Latest reading visible in Firebase/app
Detect possible fall	AI service	Valid feature vector available	Probability and class stored; alert if fall
Request SOS	Elderly user	Device online or queued	Critical SOS event generated
Assess mood	AI service	Face visible to camera	Stable emotion stored after temporal evidence
Review alert	Caregiver	Authenticated access	Severity, reason, and timestamp displayed
Review history	Caregiver	Historical records exist	Trend/event history available

Table 10 Proposed-System Use Cases

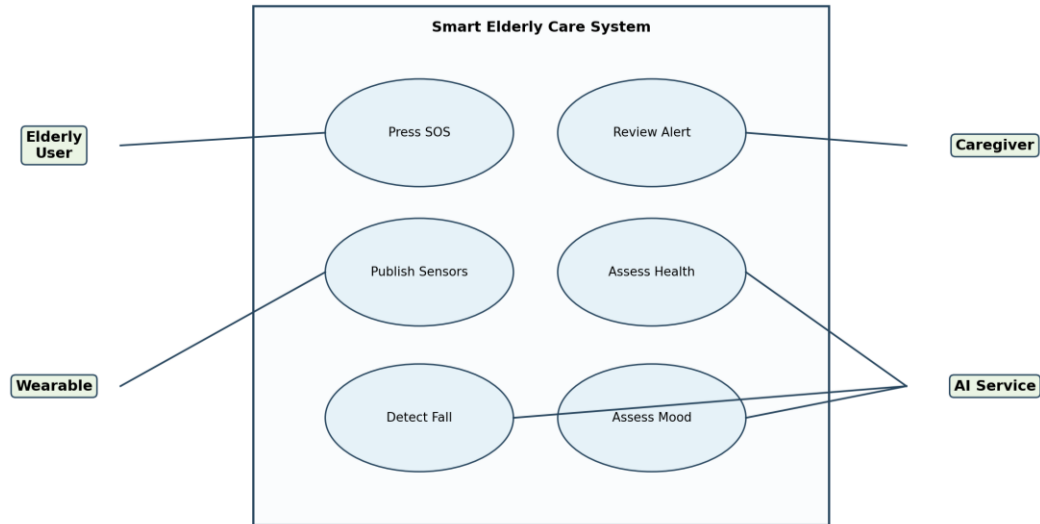


Figure 5 Proposed-system UML use-case diagram

3.4 Proposed Class Model

Class	Responsibility	Key collaborators
ElderlyCareAIEngine	Central orchestration of health, fall, mood	FallModel, HealthRiskEngine, AlertDecisionEngine
FallModel	Load bundle; validate features; infer class/probability	scikit-learn pipeline
HealthRiskEngine	Validate readings; calculate status, score, reasons, codes	HealthReading, HealthAssessment
AlertDecisionEngine	Map event codes/status to alert type and severity	HealthAssessment, AlertEvent
EmotionSmoother	Confidence filtering and temporal stability	MoodAnalysisWorker
FirebaseClient	Write latest state, history, and alert payloads	Firebase Admin SDK
MusicPlayer	Map accepted mood to local audio	pygame

Table 11 Proposed Class Model

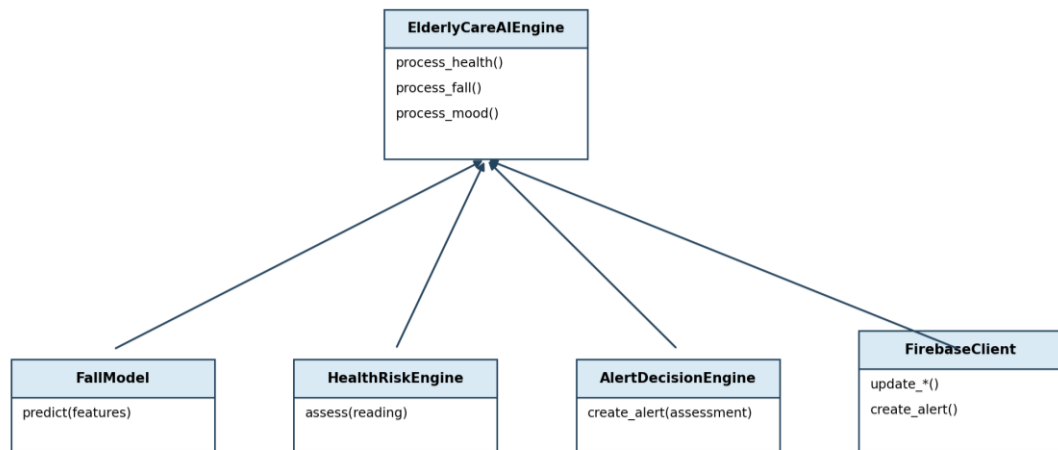


Figure 6 Core Python UML class diagram

3.5 Sequence Analysis

3.5.1 Fall Event

1. The motion window is obtained from the wearable or simulation.
2. The motion window is mapped to the precise nine-feature model.
3. FallModel verifies and makes predictions.
4. ElderlyCareAIEngine updates the fall_detected flag on the basis of that prediction.
5. HealthRiskEngine generates the CRITICAL status and FALL_DETECTED event code.
6. AlertDecisionEngine generates a critical alert.
7. FirebaseClient writes the results.
8. The caregiver app detects the change.

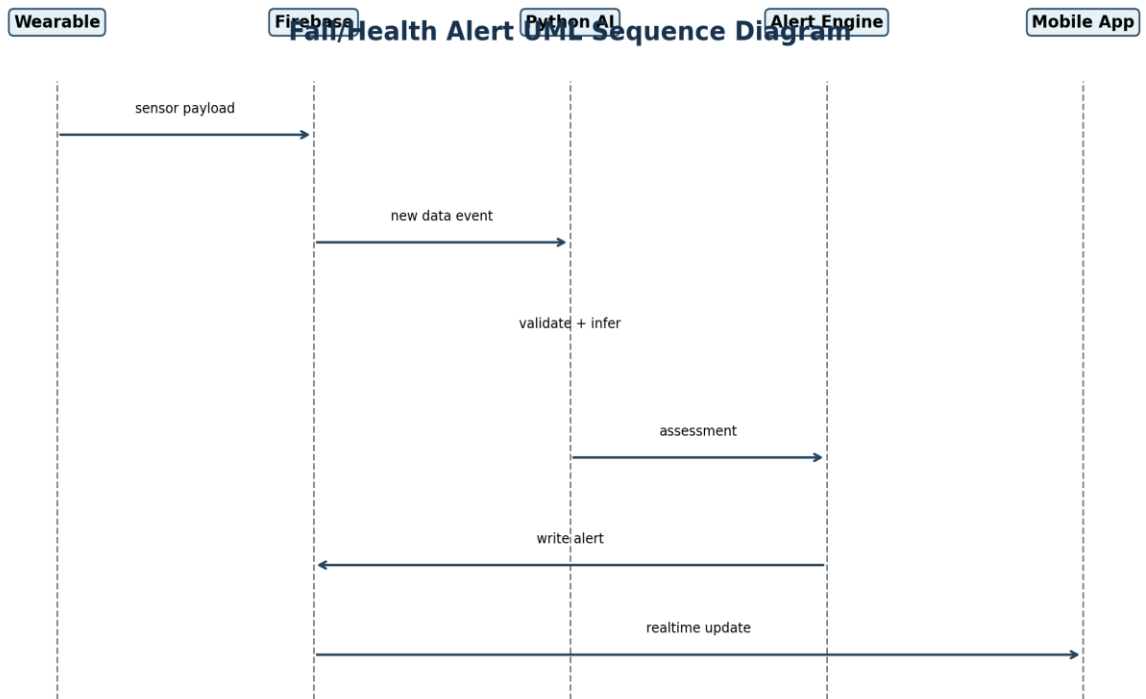


Figure 7 Fall/health alert UML sequence diagram

3.5.2 Mood Event

Mood-Analysis Pipeline with Temporal Stabilization



Figure 8 Mood-analysis sequence

Camera analysis takes place in an asynchronous manner to ensure that the frame rendering is fast. Only a new worker result is added into the smoother. Results that have low confidence levels are not considered. A candidate should remain dominant for a period of four seconds and also have three or more readings before it is classified as stable.

3.5.3 Sequence Diagrams for Each Proposed Use Case

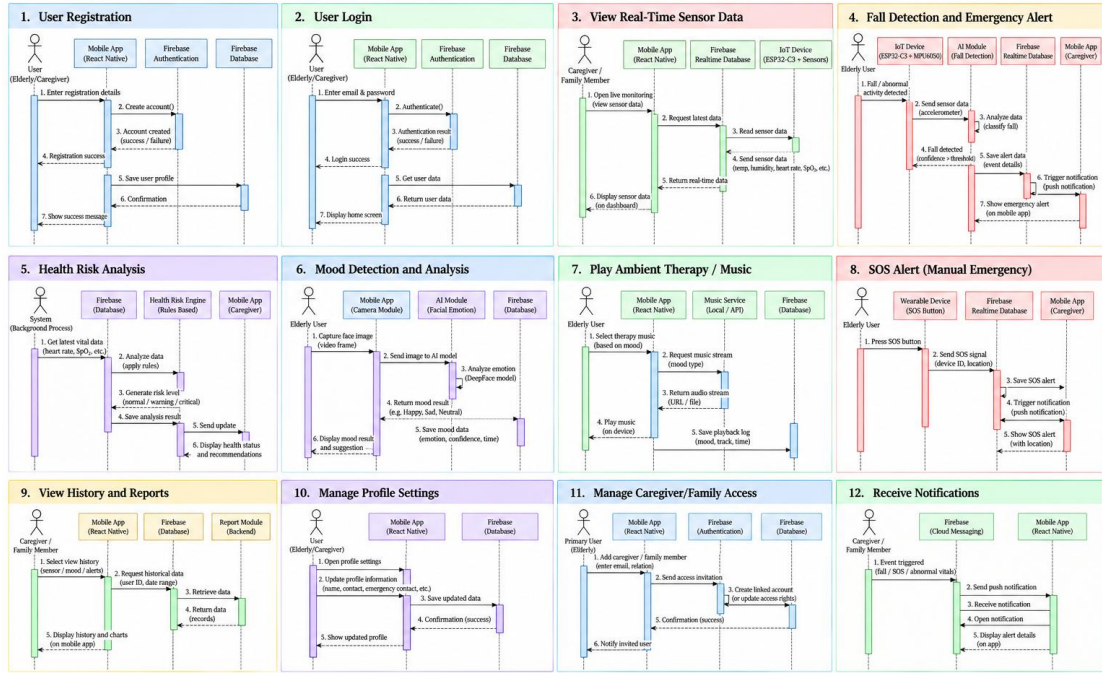


Figure 9 Sequence Diagrams for Each Proposed Use Case

Each use case has been presented through the sequence diagrams that show the interactions of the major actors and system components. The flow of sensor data, AI computation, emergency situations, mood analysis, notifications, and past history between elderly users, caregivers, IoT device, AI services, Firebase Realtime Database, and mobile application is explained here.

3.5.4 Monitor Vital Signs

The device collects physiological and physical movement data, validates the data, sends out data with timestamps to Firebase, and the mobile app displays the current state.

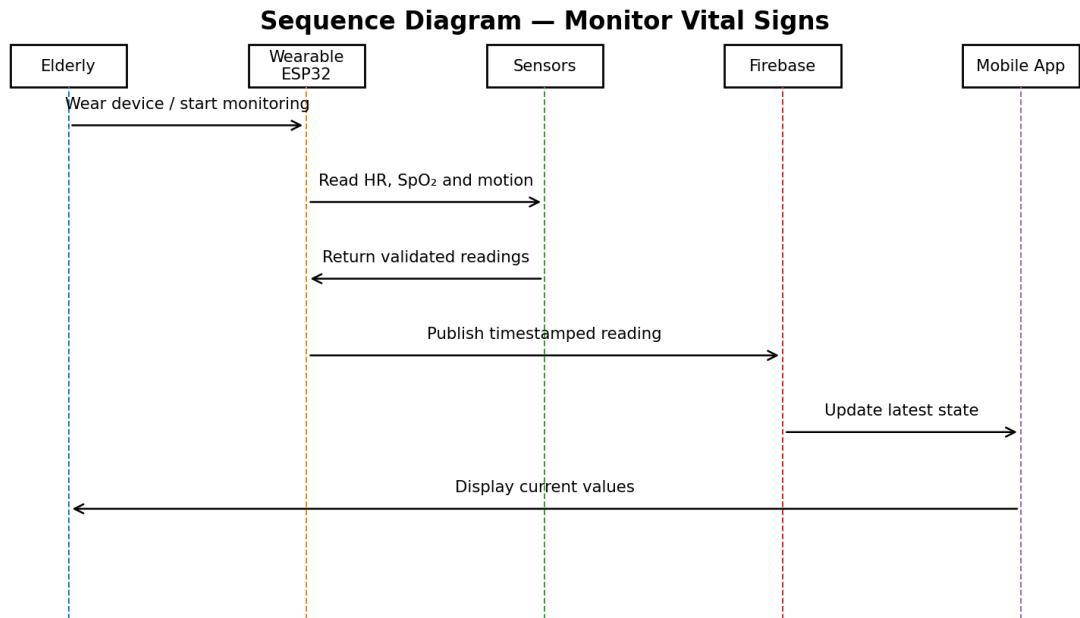


Figure 10 Sequence Diagram — Monitor Vital Signs

3.5.5 Detect Possible Fall

The AI service receives validated motion data, computes the needed feature vector, estimates the chance of a fall, and sends the information to the alert engine whenever there is a suspected fall..

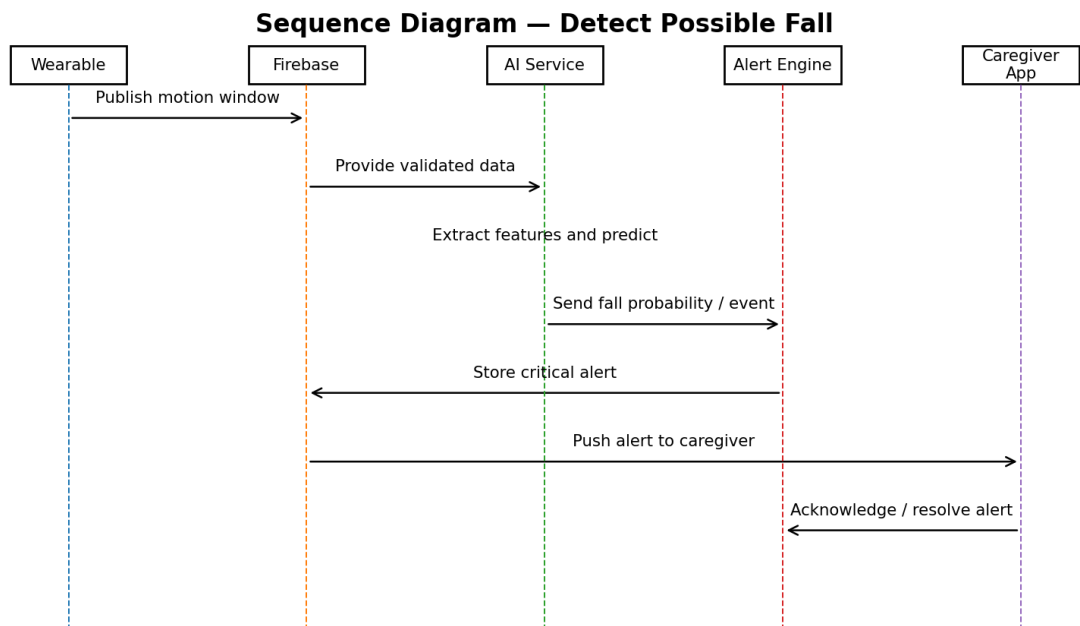


Figure 11 Sequence Diagram — Detect Possible Fall

3.5.6 Request SOS

SOS is manually initiated by the elderly person. An SOS trigger is created by ESP32, saved by Firebase, and then mapped by the alert engine to a critical caregiver alert.

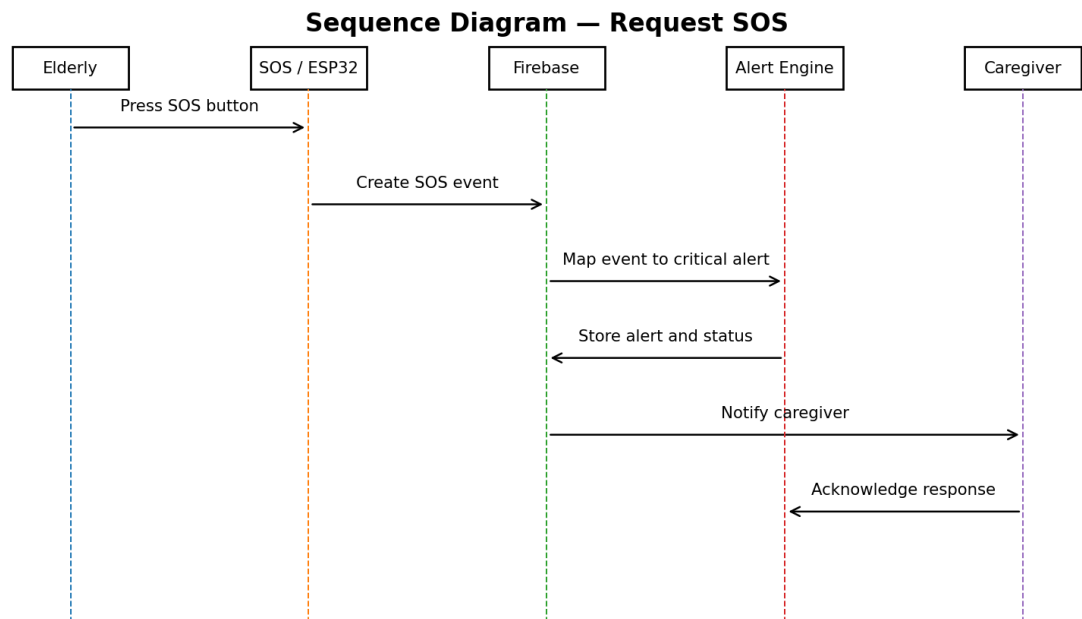


Figure 12 Sequence Diagram — Request SOS

3.5.7 Assess Mood

The camera mood system will analyze a fleeting facial expression input, validate with confidence and temporal consistency criteria, then store the accepted mood and confidence for the mobile preview system.

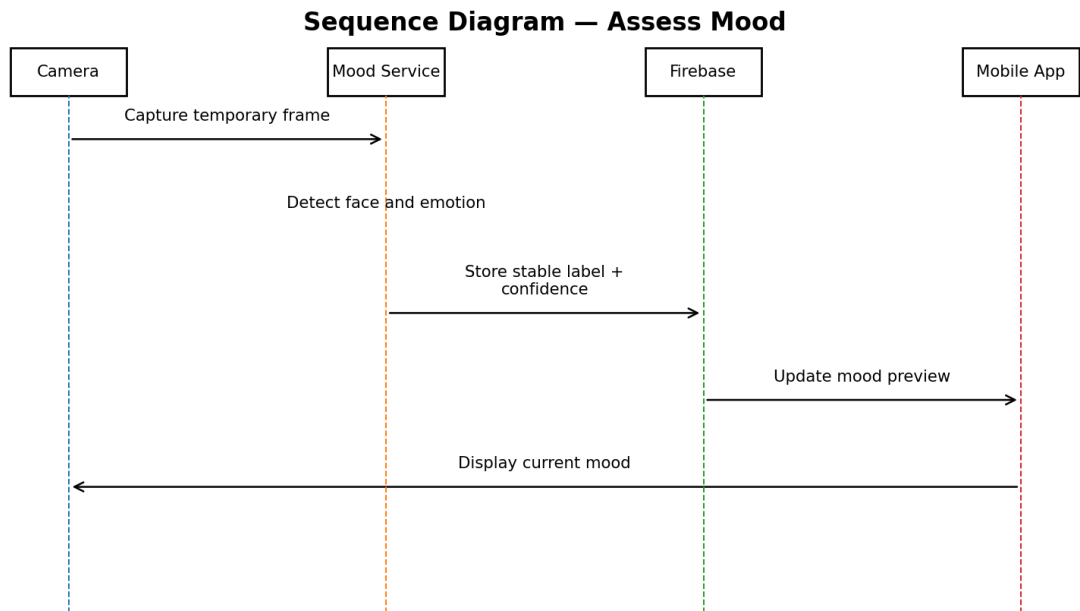


Figure 13 Sequence Diagram — Assess Mood

3.5.8 Review Alert

The legitimate caregiver launches the alert window, and the app fetches alert data from Firebase, and the caregiver acknowledges the alert.

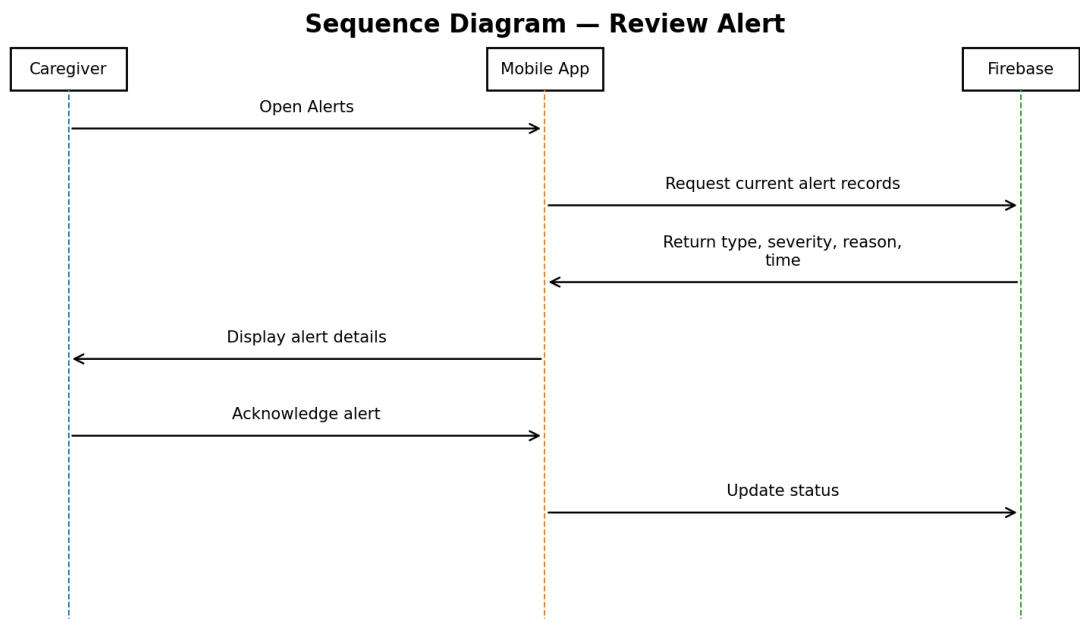


Figure 14 Sequence Diagram — Review Alert

3.5.9 Review History

The caregiver opens the history view, the application requests historical records from Firebase, and chronological readings/events are displayed.

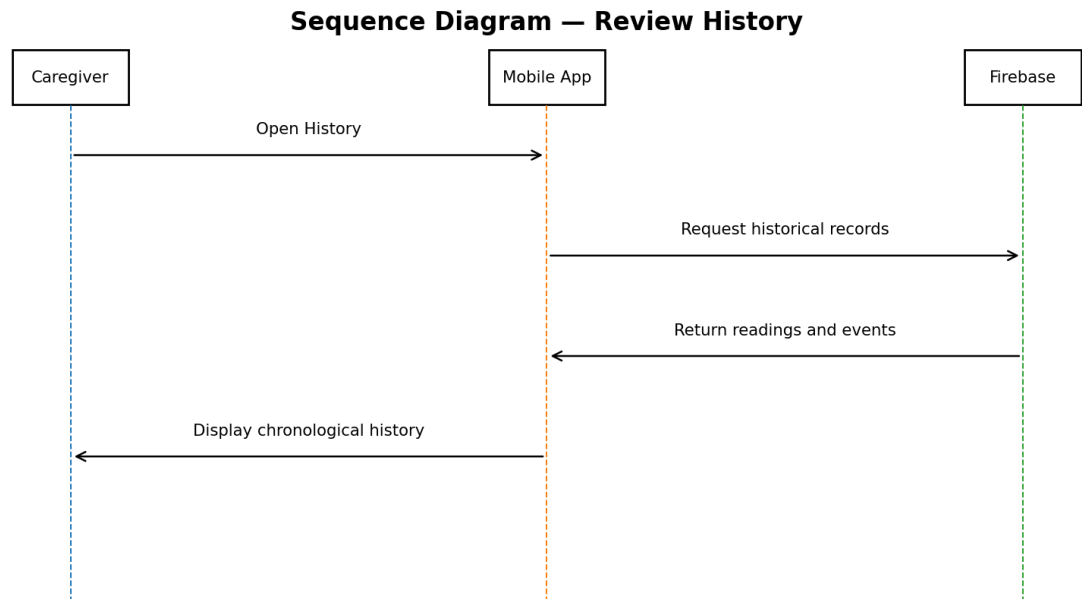


Figure 15 Sequence Diagram — Review History

3.6 Entity–Relationship and Data Analysis

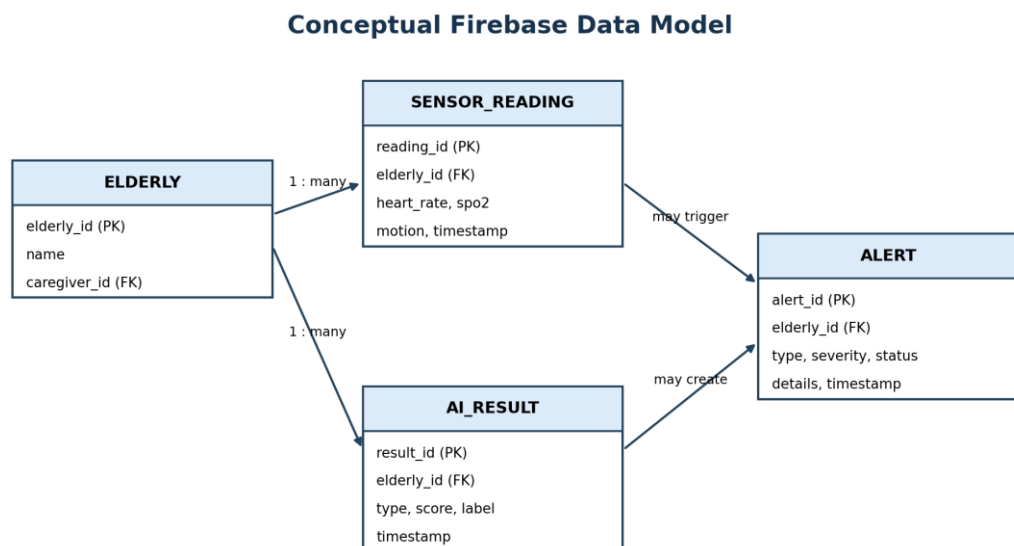


Figure 16 Conceptual data/entity model for Firebase records

Firestore is non-relational; hence, the ER diagram indicates logical ownership and references rather than SQL joins. The latest-state paths are helpful for speedy dashboard queries, whereas history and alert collections enable audit and review. Each event contains a time stamp, type, and elderly ID to help the mobile application link data records.

3.7 Functional Requirements

ID	Requirement	Verification
FR-01	Acquire motion and physiological readings	Serial output and cloud payload
FR-02	Predict fall/no-fall with probability	Simulation and model test
FR-03	Generate distinct fall and SOS alerts	Unit/integration tests
FR-04	Classify health state with explanations	Scenario simulation
FR-05	Infer and stabilize mood	Camera run and smoother tests
FR-06	Persist latest state and history	Firestore paths/local-mode payload
FR-07	Display actionable information to caregiver	Mobile interface inspection

Table 12 Functional Requirements

3.8 Non-Functional Requirements

Attribute	Requirement
Safety	Use conservative escalation and clear prototype disclaimer; never suppress SOS.
Reliability	Validate sensor ranges, feature schema, and stale readings.
Performance	Keep camera UI responsive through background analysis; use near-real-time cloud synchronization.

Security	Use authenticated access and least-privilege Firebase rules in deployment.
Maintainability	Separate sensing, model, rules, alerts, storage, and presentation modules.
Explainability	Store confidence/probability, risk score, reasons, event code, and timestamp.
Privacy	Minimize retained facial imagery; store mood labels rather than raw frames where possible.

Table 13 Non-Functional Requirements

3.9 Chapter Summary

Actors, use cases, logical classes, events sequences, data ownership, and quality requirements were analyzed. Chapter 4 translates these requirements into practical interfaces, database routes, algorithms, and deployment components..

CHAPTER 4 — SOLUTION DESIGN AND IMPLEMENTATION

4.1 Introduction

The design allows for modular borders such that the model, rules, storage adapter, device and UI could all be developed separately. The engine outputs dictionaries which are then converted by the Firebase adapter to cloud payloads.

4.2 Interface Design

Interface	Description	Primary data
I-01 Wearable acquisition	Shows/streams sensor availability and readings	Acceleration, rotation, HR, SpO ₂ , SOS
I-02 Caregiver dashboard	Summarizes current state	Status, readings, mood, connectivity
I-03 Alert detail	Supports rapid interpretation	Type, severity, message, timestamp
I-04 History	Displays past readings/events	Time-series and event records
I-05 Mood preview	Shows live and stable emotion	Label, confidence, stability state
I-06 Health monitoring	Displays detailed health measurements and trends	SpO ₂ , temperature, activity, trends
I-07 Emergency calling	Manages emergency and medical contacts and quick service calls	Contacts, phone numbers, emergency services
I-08 Caregiver profile	Provides caregiver settings and access to related records	Profile, caregiver, elderly details, settings
I-09 Help and support	Provides help topics and support contact options	Help categories, email, phone, problem report
I-10 Rules and conditions	Displays usage, health, emergency, device, notification, account and privacy rules	Rules, privacy, acknowledgement

Table 14 Interface Design

I-02 — Caregiver Dashboard: Normal State

The dashboard state will show the current health state, physical activity, mood state, environment sensors readings, synchronization state, and alerts generated in case the overall safety state is in good condition.

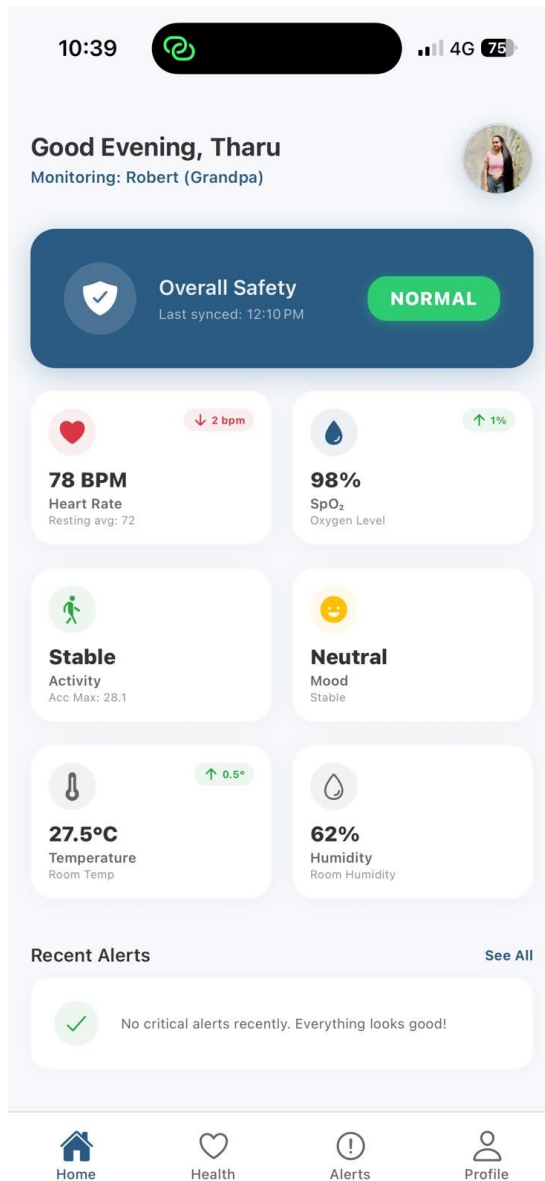


Figure 17 Caregiver Dashboard — Normal State

I-02 — Caregiver Dashboard: Critical State

In this state of the dashboard, there is an emphasis on the safety status, along with the information on the fall alert, which was recently issued.

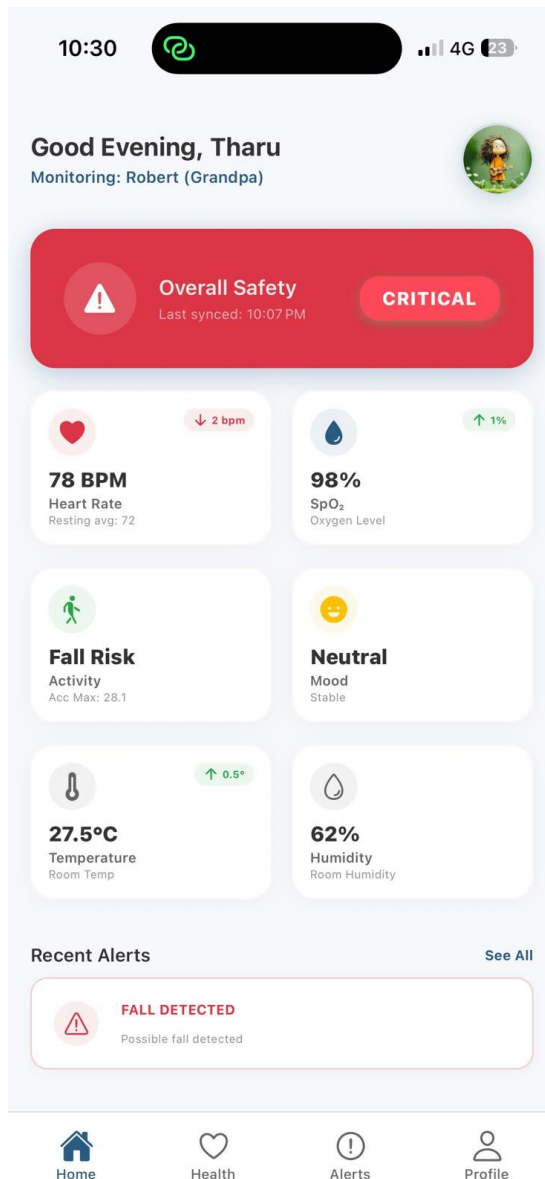


Figure 18 Caregiver Dashboard — Critical State

I-05 — Mood Auxiliary Tracking Interface

This interface shows the currently identified mood, confidence, time stamp, and mood categories. This is shown as an additional trend indicator and not as a diagnosis.

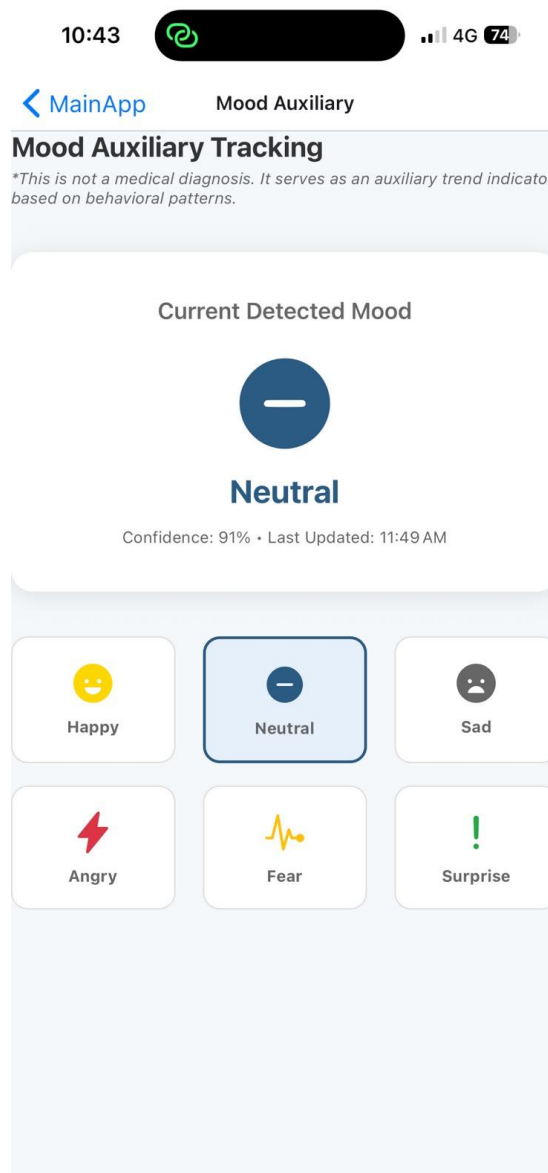


Figure 19 Mood Auxiliary Tracking Interface

I-03/I-04 — Emergency Alerts and History Interface

This interface shows information regarding emergencies, timestamps, status of resolution, caregiver response data, and access to history of alerts.

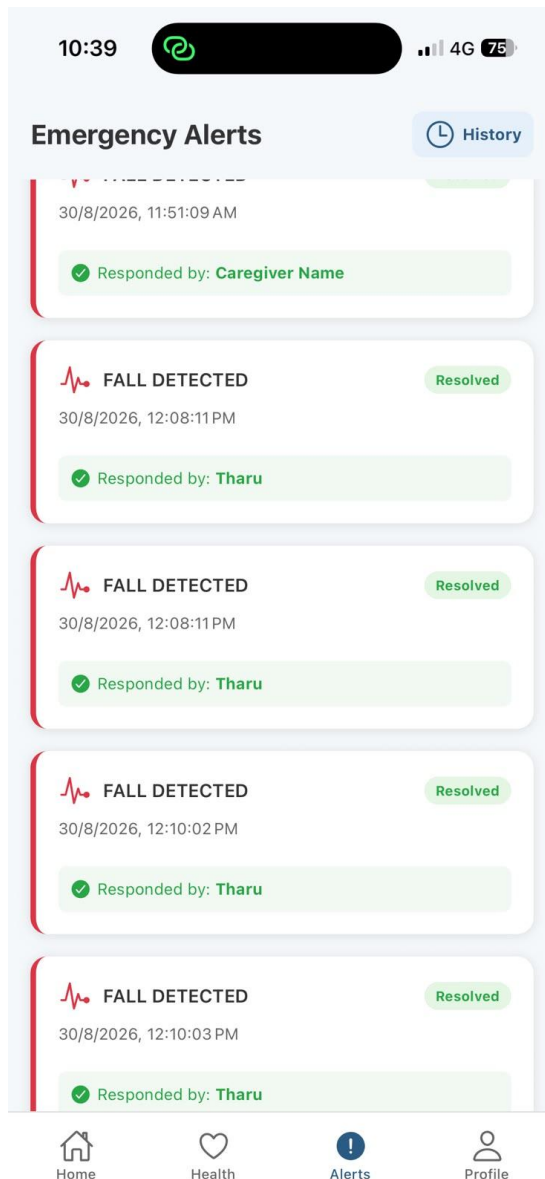


Figure 20 Emergency Alerts and History Interface

I-06 — Health Monitoring Interface

This is the screen which shows the health parameters, such as SpO₂, body temperature, and activity status.

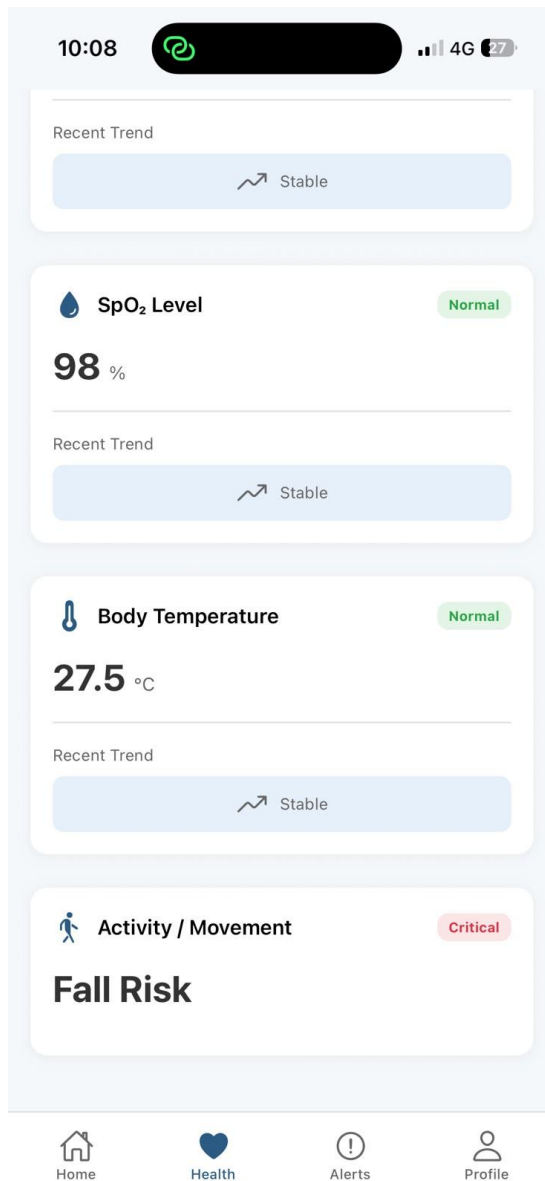


Figure 21 Health Monitoring Interface

I-07 — Emergency Calling Interface

It helps maintain emergency contacts as well as medical contacts and facilitates instant contact with ambulance, police, and fire and rescue services.

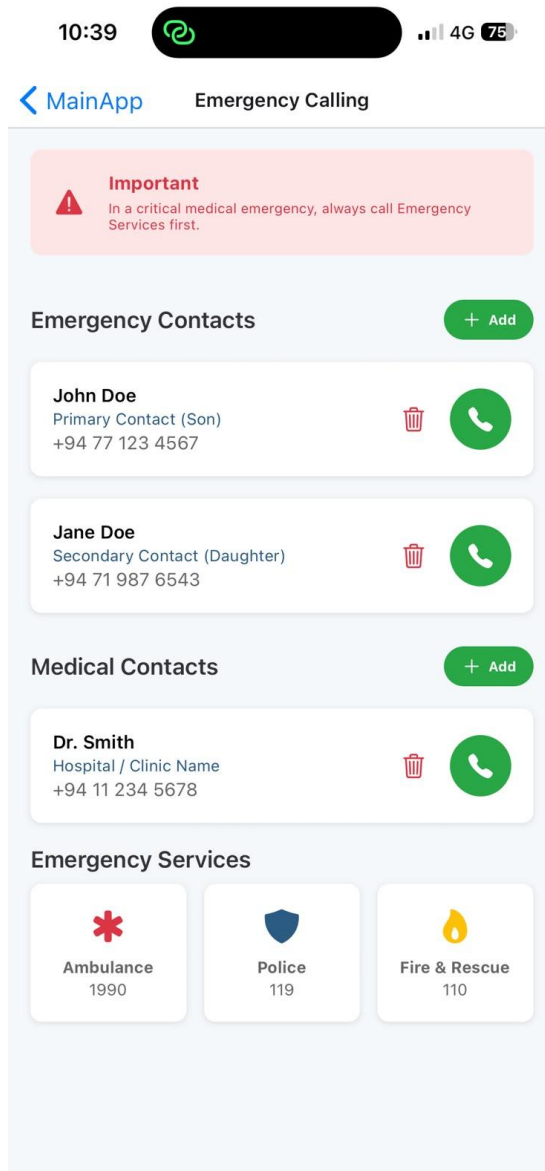


Figure 22 Emergency Calling Interface

I-08 — Caregiver Profile Interface

The user can get access through this interface to caregiver information, reports and summaries, caregiver and elder person information, emergency call, push notification, and alert sound.

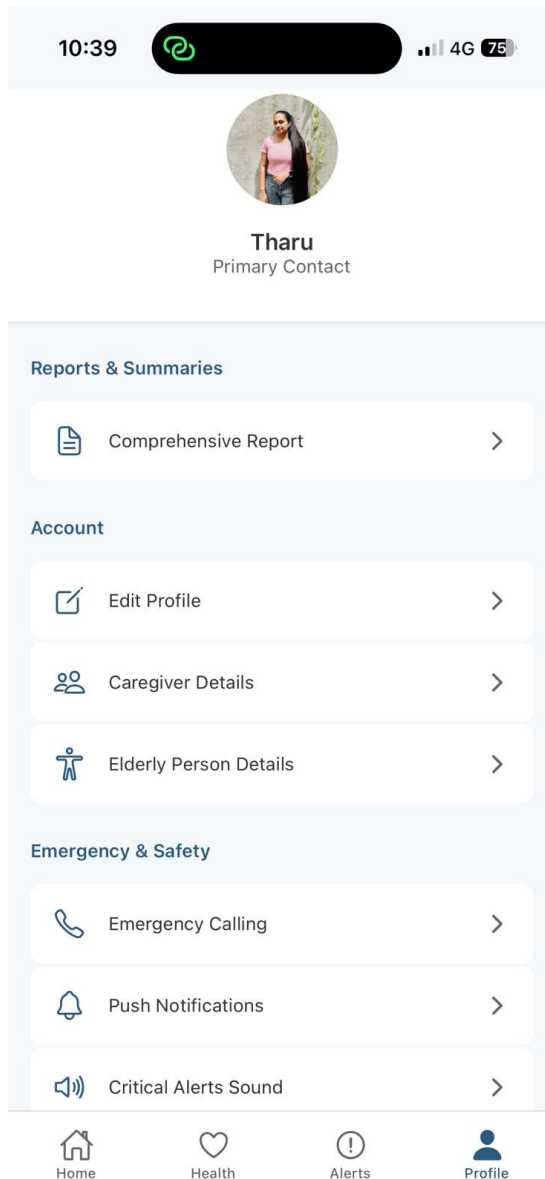


Figure 23*Caregiver Profile Interface*

I-09 — Help and Support Interface

The interface offers help in categories that include starting out, health monitoring, emergencies, device assistance, and account assistance, among others.

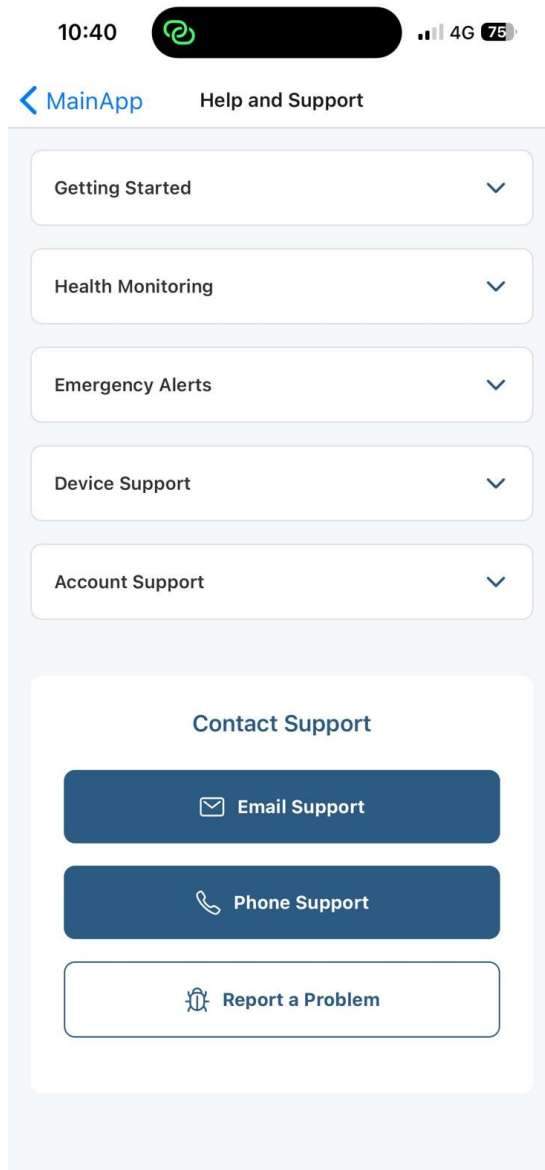


Figure 24 *Help and Support Interface*

I-10 — Rules and Conditions Interface

The following are the guidelines that cover usage, health monitoring, emergency alerts, device, notifications, account, data and privacy, and acknowledgement for the caregiver app.

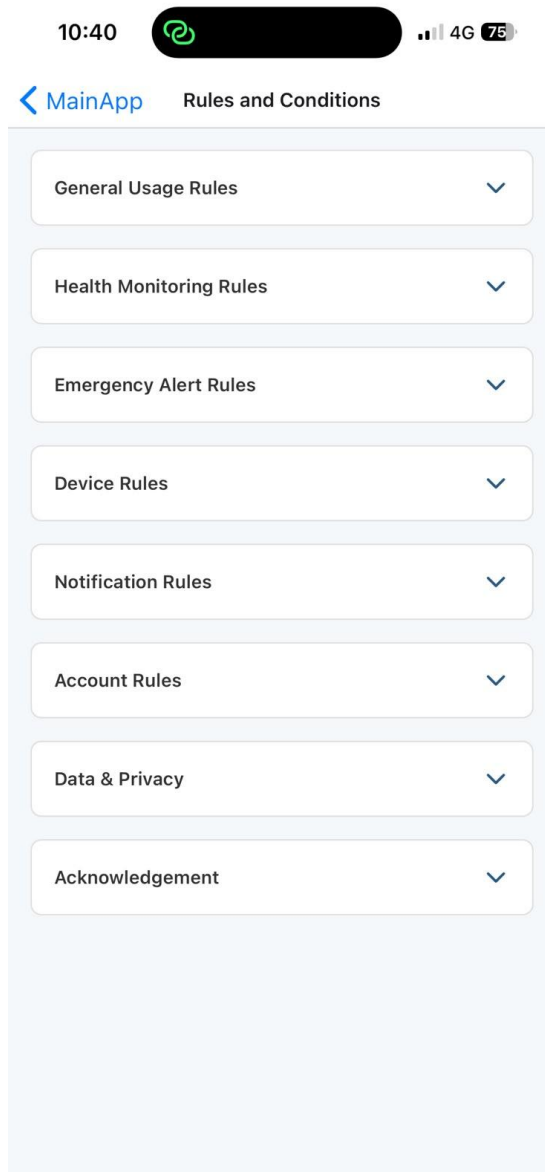


Figure 25 Rules and Conditions Interface

4.2.1 Interface Pseudocode

Wearable loop pseudocode: initialize sensors and WiFi; loop over MPU6050 and MAX30102 reads; confirm finger/contact; read SOS; create timestamped message; push the most recent value to Firebase; add event when needed; retry after a failed network operation.

AI listener pseudocode: check for new Firebase/device data; confirm the identity and schema; collect motion window; extract feature vector; invoke stored Random Forest pipeline; send output to system engine; post prediction, health evaluation, and any alarm.

Caregiver pseudocode: authenticate; subscribe to elderly/latest and elderly/alerts; display current status; stress warning/critical severity; enable alert acknowledge; open history when desired.

4.2.2 Interface-wise Pseudocode

I-01 — Wearable Acquisition

INITIALIZE ESP32, MPU6050, MAX30102 and Wi-Fi

LOOP

 READ sensor values

 VALIDATE contact and ranges

 READ SOS input

 ADD timestamp

 PUBLISH latest reading to Firebase

 IF event exists THEN append event

 IF network fails THEN retry safely

END LOOP

I-02 — Caregiver Dashboard

AUTHENTICATE caregiver

SUBSCRIBE to latest state and alerts

READ health status, HR, SpO₂, activity, mood and environment

IF severity is Critical THEN highlight critical state

ELSE display Normal or Warning state

UPDATE dashboard when Firebase data changes

I-03 — Alert Detail

SELECT alert from alert list

READ type, severity, reason and timestamp

DISPLAY alert details

IF caregiver acknowledges THEN update alert status
STORE response information

I-04 — History

AUTHENTICATE caregiver
REQUEST historical records
SORT records by timestamp
DISPLAY readings and events chronologically
ALLOW caregiver to review previous alerts

I-05 — Mood Preview

RECEIVE mood label and confidence
CHECK confidence threshold
APPLY temporal stability rule
IF stable THEN display accepted mood
ELSE keep previous stable state
DISPLAY confidence and update time

I-06 — Health Monitoring

REQUEST latest health measurements
DISPLAY SpO₂, temperature and activity
CALCULATE / READ trend state
DISPLAY Normal, Warning or Critical status
REFRESH when new valid readings arrive

I-07 — Emergency Calling

LOAD emergency and medical contacts
DISPLAY saved contacts and service numbers
IF user selects contact THEN initiate phone call
IF user adds contact THEN validate and save contact
IF user deletes contact THEN remove selected contact

I-08 — Caregiver Profile

LOAD authenticated caregiver profile
DISPLAY caregiver and elderly details

IF Edit Profile selected THEN update allowed fields

DISPLAY links to reports, emergency settings and notifications

I-09 — Help and Support

DISPLAY help categories

IF category selected THEN expand support instructions

IF Email or Phone selected THEN open contact action

IF Report Problem selected THEN open problem-report workflow

I-10 — Rules and Conditions

DISPLAY rule categories

IF category selected THEN expand rules

DISPLAY health, emergency, device, notification, account and privacy rules

IF acknowledgement selected THEN record acknowledgement

The below table represents the logical representation of the mapping of the Firebase data structure to the fields of the guidelines. As a prototype uses Firebase Realtime Database and not the relational DBMS, the Firebase identifier/path key represents the primary key, whereas the elderly identifier represents the logical relationship key.

Table No.	Table Name	Primary Key	Foreign Key	Description
T-01	Elderly	elderly_id	—	Stores elderly profile and identity information.
T-02	Sensor Reading	reading_id	elderly_id	Stores latest heart rate, SpO ₂ , motion and timestamp.
T-03	Fall Result	fall_id	elderly_id	Stores fall/no-fall prediction, probability

				and timestamp.
T-04	Mood Result	mood_id	elderly_id	Stores stable mood label, confidence, model and timestamp.
T-05	Health Assessment	assessment_id	elderly_id	Stores status, risk score, reasons, event codes and timestamp.
T-06	Alert / History	event_id	elderly_id	Stores actionable alerts and historical event records with status and timestamp.

Table 15 logical relationship key

4.3 Database Design

Path / collection	Key fields	Purpose
elderly/{id}/sensors/latest	heart_rate, spo2, motion, timestamp	Current dashboard state
elderly/{id}/ai/fall	prediction, fall_probability, timestamp	Latest model result
elderly/{id}/ai/mood	emotion, confidence, backend, model, timestamp	Latest stable mood

elderly/{id}/ai/health	status, risk_score, reasons, event_codes, timestamp	Latest explainable assessment
elderly/{id}/alerts/{pushId} }	type, severity, status, details, timestamp	Append-only actionable alerts
elderly/{id}/history/{type}/ {pushId}	payload, timestamp	Historical audit and trends

Table 16 Database Design

4.4 Fall Detection Design

The reason for selecting the Random Forest algorithm was that the algorithm can handle nonlinear boundaries, mixed scales of features, strong tabular data performance, and probability outputs. Each tree votes from a bootstrap/sample of the features point of view; by aggregation, the variance of an individual decision tree can be reduced.

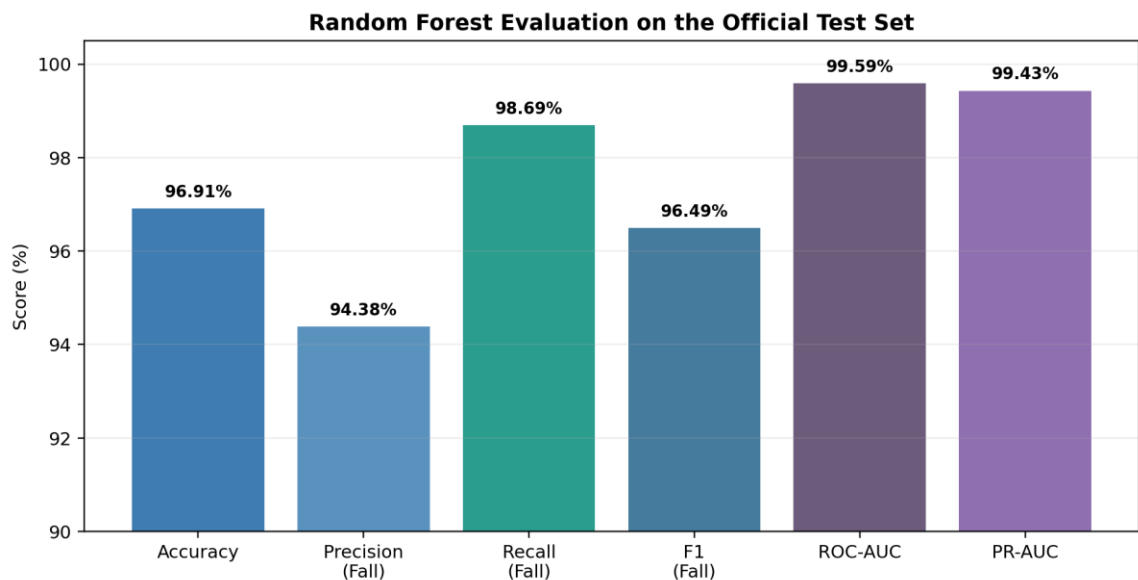


Figure 26 Held-out fall-model evaluation metrics

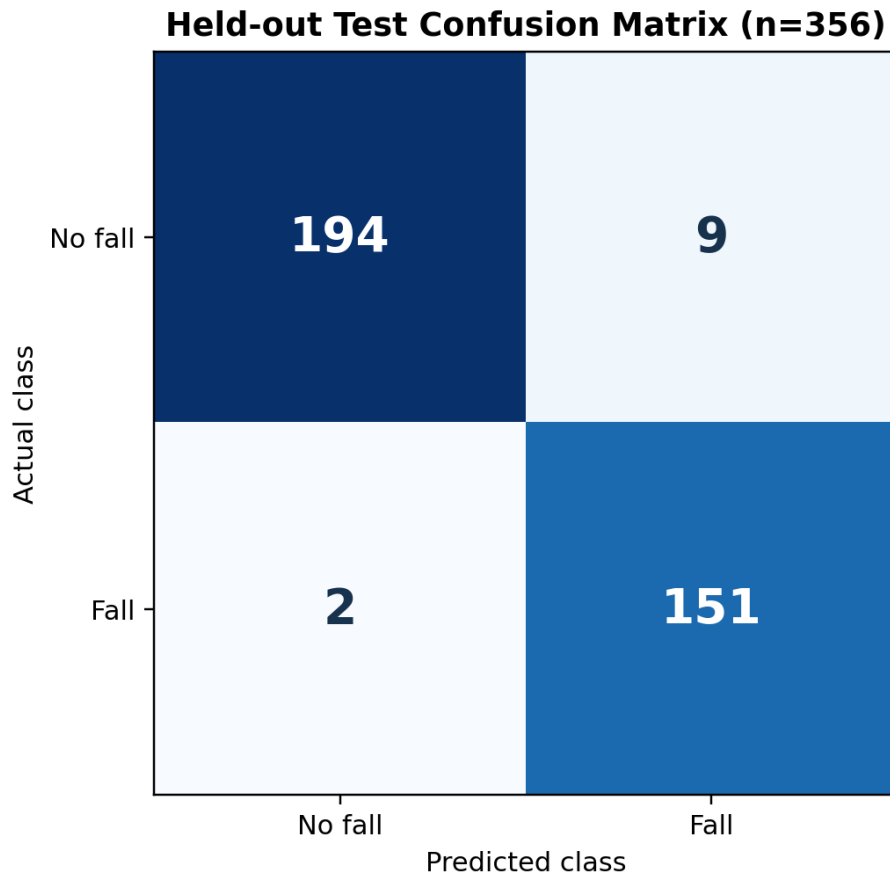


Figure 27 Held-out confusion matrix

From 153 real cases of falls, 151 instances were identified while 2 went undetected, thus resulting in 98.69% recall. From 203 cases of no falls, 9 instances were falsely indicated as being falls, resulting in 94.38% precision. In this setting where there is emphasis on safety, a higher recall was preferred since the consequence of missing a fall can be more damaging than generating a false positive.

Metric	Value	Interpretation
Cross-validation F1	97.57%	Strong mean training-fold generalization
Test accuracy	96.91%	345 of 356 test decisions correct
Fall precision	94.38%	Most fall alerts corresponded to falls
Fall recall	98.69%	151 of 153 falls detected
Fall F1	96.49%	Balanced fall-class performance

ROC-AUC	99.59%	Strong ranking across thresholds
PR-AUC	99.43%	Strong positive-class precision–recall trade-off

Table 17 metric value

4.5 Health-Risk and Alert Design

Rule-based, as opposed to learning-based, is the design choice made for the health module. First, this module eliminates impossible sensor readings, and next, considers SOS, fall, SpO₂, resting heart rate, and inactivity. Critical incidents supersede any warnings built up from analysis. Its output consists of status, risk level, causes, requires_alert, and event_codes. Using event_codes makes it possible to distinguish between, for example, SOS_EMERGENCY or FALL_DETECTED and simply HEALTH_CRITICAL.

Condition	Decision	Alert code
SOS pressed	Critical, risk 100	SOS_EMERGENCY
Fall detected	Critical, risk 100	FALL_DETECTED
SpO ₂ ≤ 92% demo threshold	Critical, risk 100	LOW_SPO2_CRITICAL
92% < SpO ₂ < 95%	Warning contribution	LOW_SPO2_WARNING
HR outside 60–100 bpm demo range	Warning contribution	HEART_RATE_WARNING
Inactivity ≥ 60 min	Warning contribution	INACTIVITY_WARNING
Invalid reading	Warning	SENSOR_WARNING

Table 18 Health-Risk and Alert Design

4.6 Mood Analysis Design

Mood detection is performed using a module which captures webcam snapshots using OpenCV, detects faces using RetinaFace and applies DeepFace's pre-trained emotion detection. A separate worker thread is used to prevent blocking of the video feed due to the relatively costly process of inference by the model. The temporal smoothing is weighted with confidence and takes into account continuous information only: confidence less than 40% is discarded; it takes four seconds and three reads to achieve a steady label; a gap between detections resets the candidate.

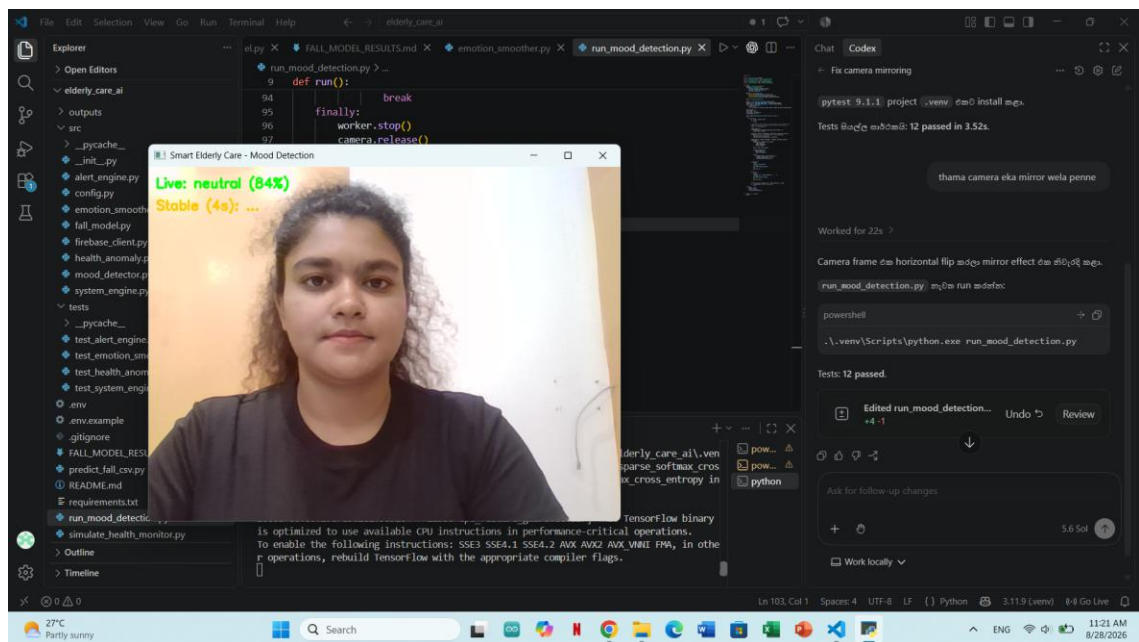


Figure 28 Live mood-detection interface

4.7 IoT and Cloud Integration

The initialization of the sensors MPU6050 and MAX30102, setting up the connection to the Wi-Fi network, and publishing sensor data together with timestamps in Firebase Realtime Database are performed by ESP32-C3. Since the sensor MAX30102 demands continuous finger contact for proper physiologic data collection, the non-occurrence of finger contact is denoted by invalid data instead of zero physiologic data values. Firebase Realtime Database is used as a shared state layer between ESP32-C3, AI services (Python), and mobile application of the caregiver.

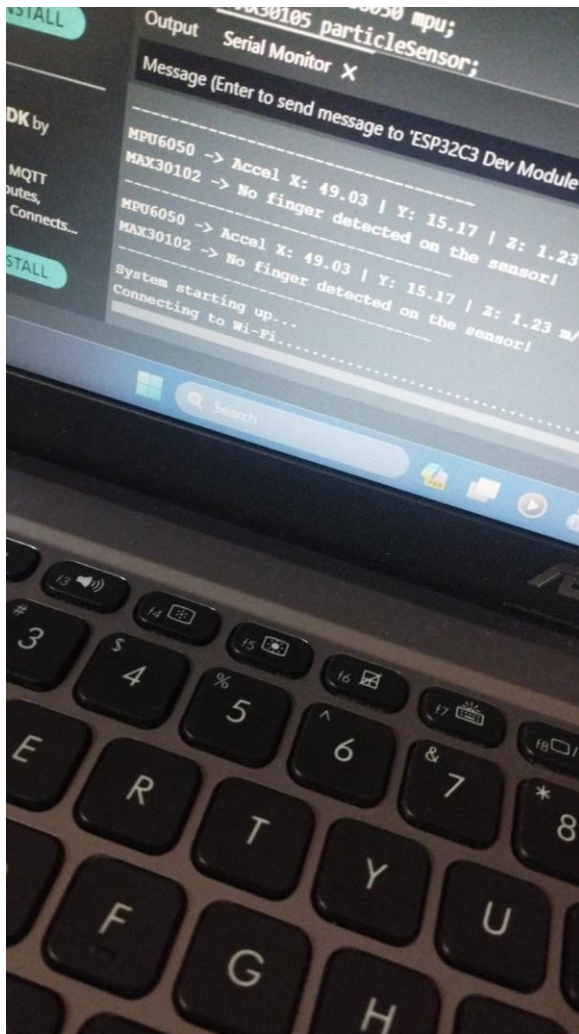


Figure 29 Wearable sensor serial output during integration

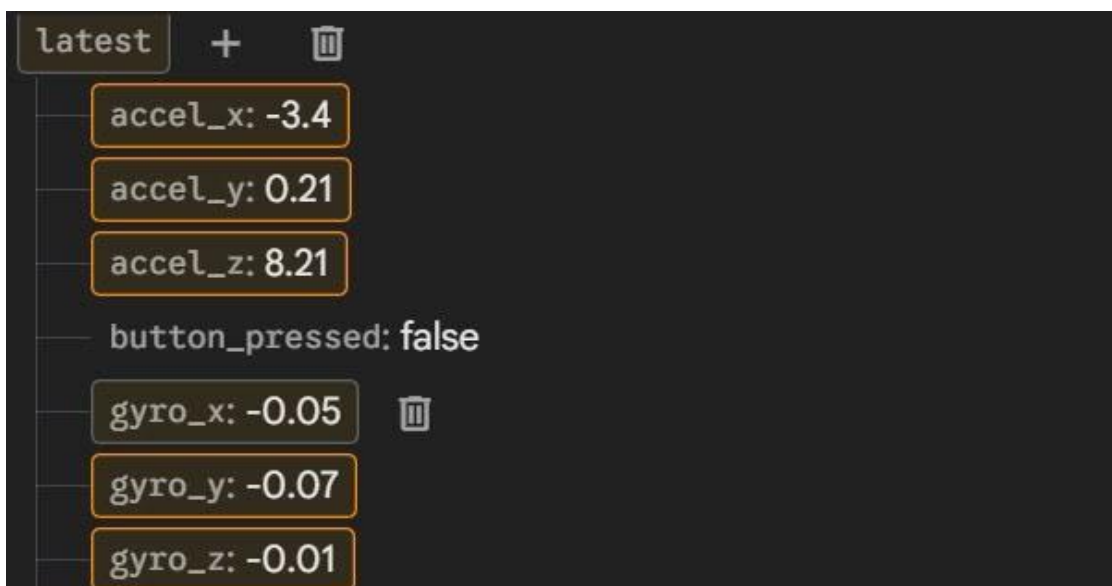


Figure 30 Firebase Realtime Database — Latest Motion and SOS Data

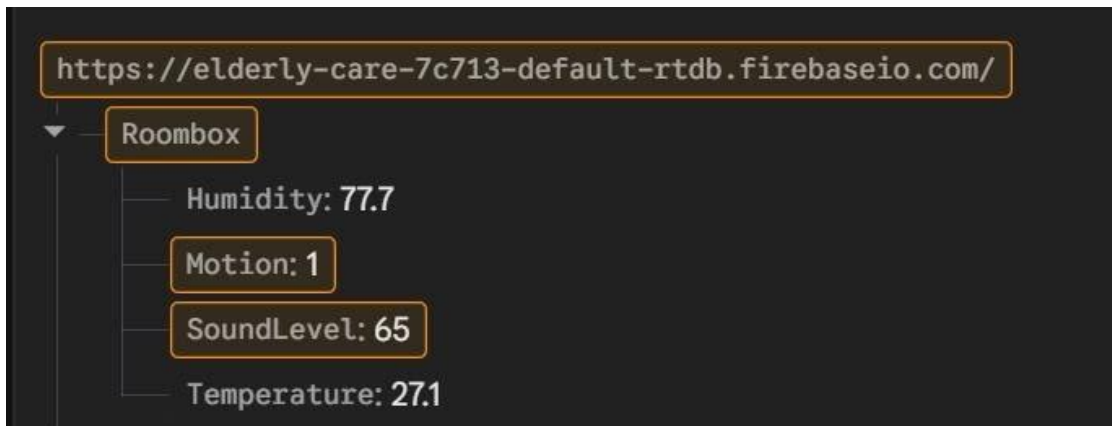


Figure 31 Firebase Realtime Database — Environmental Sensor Data

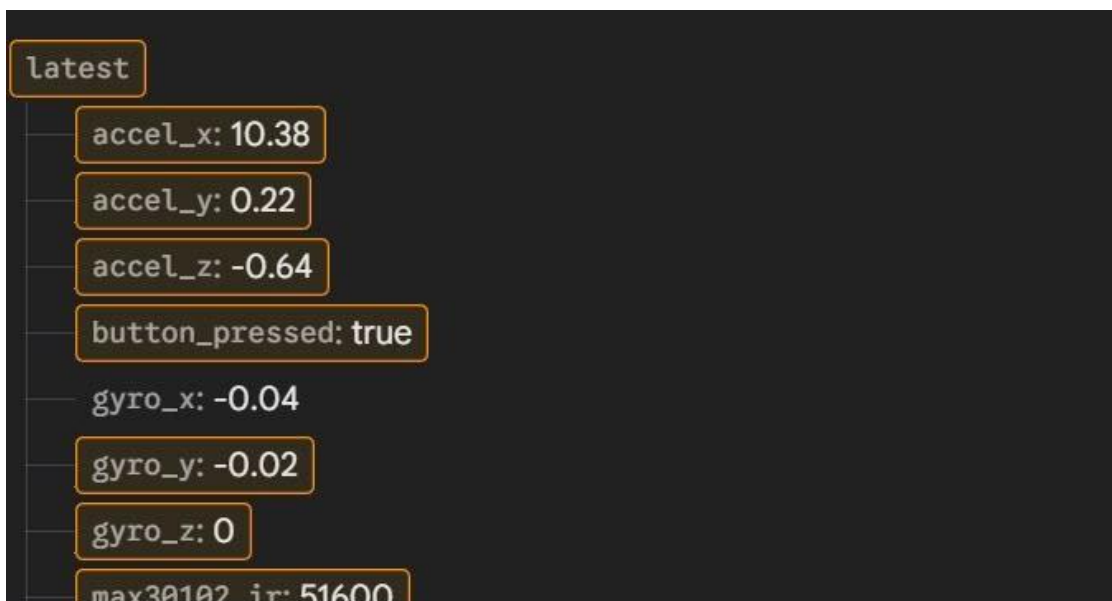


Figure 32 Firebase Realtime Database — Accelerometer and Gyroscope Data

4.8 Report Layout Design

Report Number	Layout	Report Name	Description
RL-01		Comprehensive Care Report	Provides a consolidated overview of elderly, caregiver, and medical/doctor details.
RL-02		Current Health Report	Displays HR, SpO ₂ , status, connectivity and timestamp.
RL-03		Critical Alert Report	Displays alert type, severity, reason, elderly identity and time.
RL-04		Fall Prediction Report	Displays fall class and prediction probability.
RL-05		Mood Status Report	Displays stable emotion and confidence.
RL-06		History Report	Displays chronological readings and resolved alerts.
RL-07		Model Evaluation Report	Displays dataset, evaluation metrics, confusion matrix and limitations.

Table 19 Report Layout Design

RL-01 — Comprehensive Care Report

This structure of the report enables us to have all the information related to the old individual, the caregiver, and the physician in one place.

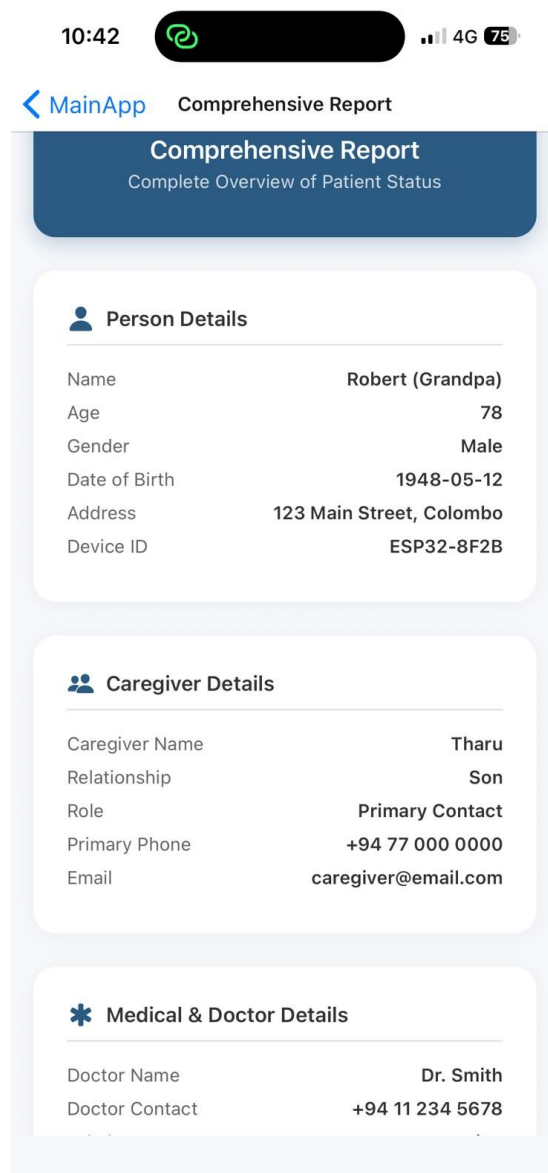


Figure 33 Comprehensive Care Report Interface

4.9 Security, Privacy, and Safety Controls

Limit the Firebase read and write access based on authenticated user and assigned elderly ID.

Do not store credentials in source code and change any exposed key.

Check payload type, range, timestamp, and required fields prior to processing.

Keep facial data minimal and never store the camera footage unless required.

Log the status of an alert and timestamps for tracking.

Provide a manual option to initiate an SOS irrespective of model accuracy.

Consider the system assistive monitoring only; guide the caregivers to validate and act accordingly.

4.10 Implementation Evidence

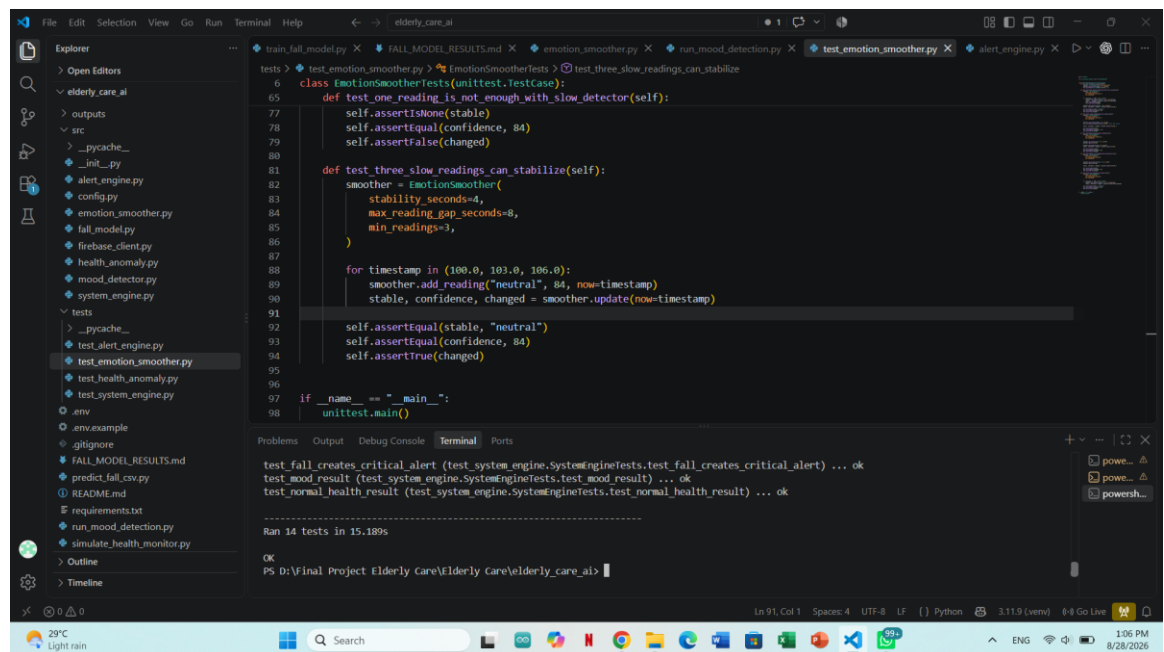


Figure 34 Automated test suite completion evidence

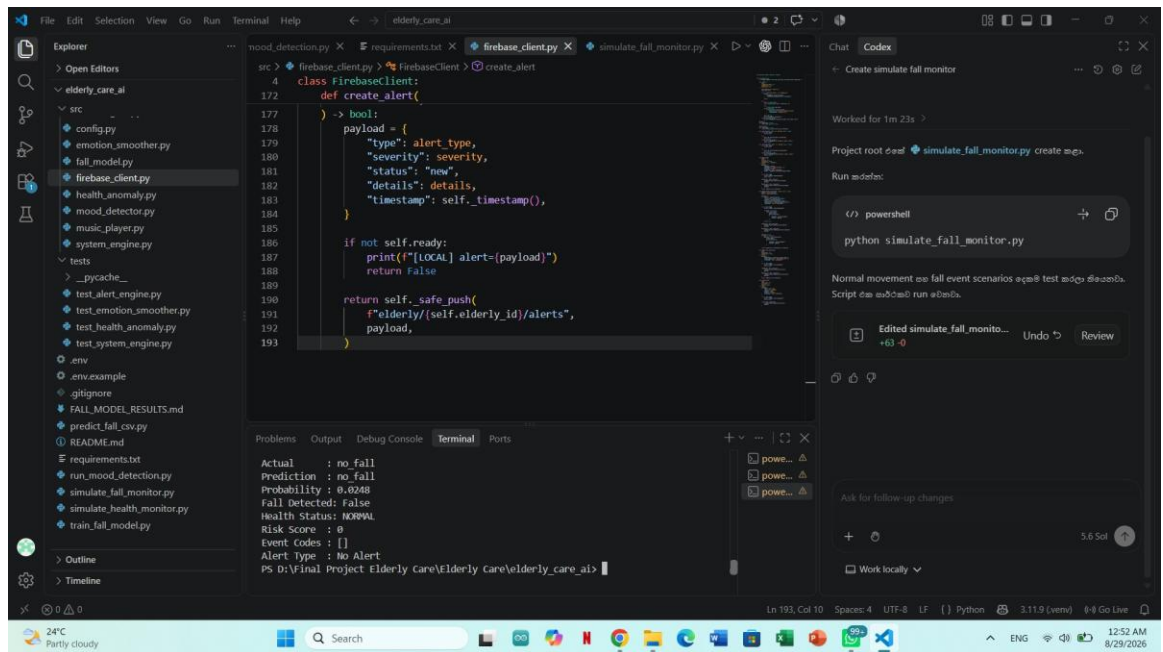


Figure 35 Fall/no-fall simulation evidence

4.11 Chapter Summary

The final implemented design incorporates modeling inferences, deterministic safety rules, temporal smoothing of emotions, cloud-based storage and alerting to users. The following chapter considers accomplishments, limitations, and further improvements.

CHAPTER 5 — CONCLUSION

5.1 Achievement Against Objectives

Objective	Evidence	Status
Wearable sensor prototype	ESP32-C3 serial output with MPU6050/MAX30102 and Wi-Fi startup	Achieved at prototype level
Fall AI model	Held-out metrics and serialized pipeline	Achieved on public dataset
Health-risk reasoning	Normal/warning/critical/fall/SOS simulations	Achieved
Mood recognition	Live camera inference, stabilization, Firebase payload, music	Achieved
Cloud integration	Firebase client methods for mood, health, fall, alerts/history	Achieved; configuration-dependent
Verification	18 automated tests passed in final project state	Achieved

Table 20 Achievement Against Objectives

5.2 Model Evaluation and Interpretation

The key metric from the fall model that has safety implications is recall with a score of 98.69% on the official test set, equivalent to two false negatives. The 94.38% precision score is equivalent to nine false positives. It is only with the aid of metrics such as F1, ROC-AUC, and PR-AUC in addition to the confusion matrix that this disparity becomes apparent. The cross-validation F1 score of 97.57% correlates well with the held-out F1 score of 96.49%, indicating selection optimism, but not too much of it.

5.3 Software Verification

The test suite included tests for alert creation, health status checks (normal, warning, and critical), low confidence and stale mood data, long delays between detections, three-readings to stabilize, system-wide falls alert, mood result, and normal health

status result. More recent additions include tests for event-specific falls and SOS mapping. These passing tests ensure correct functioning of the code for certain situations but cannot substitute field testing.

5.4 Strengths

Combination of AI, IoT, cloud, and mobile design instead of the standalone approach.

Highest recall on test data with clearly provided confusion matrix.

Replicable workflow with pre-processing and classification stages.

Explanations of the results with causes and event codes.

Temporal mood filtering for stable frame-to-frame predictions.

Firebase locally available as a fallback for safe testing without credentials.

Modular source code and regression tests.

5.5 Limitations and Threats to Validity

Limitation	Impact	Mitigation / future work
Smartphone-derived engineered features	Sensor placement and distributions may differ from MPU6050 wearable	Collect synchronized device data and retrain/calibrate
Unknown authoritative feature-window formulas	Cannot guarantee live feature equivalence	Obtain raw dataset protocol or define and validate a new extractor
No prospective elderly cohort	Performance may not transfer to target population	Ethics-approved pilot with diverse users
Pretrained facial emotion model	Bias, lighting, pose, and cultural expression can affect labels	Treat as optional wellness cue; consent and subgroup testing

Demo health thresholds	Not personalized or clinician-approved	Clinical governance and personalized baselines
Internet/cloud dependency	Delayed updates during outage	Local buffering, reconnection, offline alarm path
Prototype security evidence	Rules and credential lifecycle not fully audited	Security testing and least-privilege rules

Table 21 Limitations and Threats to Validity

5.6 Recommended Future Work

Specify the sampling rate of the MPU6050, the motion window duration, filter type, and nine features extraction.

Capture labelled ADLs and simulated falls according to safety measures and ethical considerations.

Run subject-based cross validation and external validation of the device to avoid person-level leakages.

Configure decision threshold based on clear definitions of missed falls and false alarms.

Consider monitoring the device's battery, connection, sensor quality, and outdated data.

Design acknowledgement, escalation timeout, and caregiver audit logging mechanisms.

Evaluate accessibility and usability by older persons and caregivers.

Conduct privacy impact assessment, Firebase rules, penetration, and operation failure testings.

5.7 Conclusion

The project provided a unified smart elderly care prototype combining wearable sensors, an advanced Random Forest fall classifier, an explainable risk of health logic, a stable emotion recognition using cameras, Firebase data transfer, mood-based music generation, and caregiver outputs. The most convincing technical proof-of-concept presented by the project is a fall assessment on holdout data and an automated test suite. The most significant engineering lesson learned from the project is that good software is not enough to build a good product but data contract, temporal validation, event-specific alerts, failures management, and realistic boundaries of implementation are also essential. This prototype can be used for academic demonstrations and further research but it cannot be considered a medical device yet.

References

- Breiman, L. (2001) 'Random forests', Machine Learning, 45, pp. 5–32.
- Deng, J., Guo, J., Zhou, Y., Yu, J., Kotsia, I. and Zafeiriou, S. (2020) 'RetinaFace: Single-shot multi-level face localisation in the wild', Proceedings of CVPR.
- Espressif Systems (2026) ESP32-C3 Series Datasheet. Available at: <https://www.espressif.com/> (Accessed: 3 September 2026).
- Firebase (2026) Firebase Realtime Database documentation. Available at: <https://firebase.google.com/docs/database> (Accessed: 3 September 2026).
- OpenCV (2026) OpenCV documentation. Available at: <https://docs.opencv.org/> (Accessed: 3 September 2026).
- scikit-learn developers (2026) RandomForestClassifier and RandomizedSearchCV documentation. Available at: <https://scikit-learn.org/stable/> (Accessed: 3 September 2026).
- Serengil, S.I. and Ozpinar, A. (2021) 'LightFace: A hybrid deep face recognition framework', Innovations in Intelligent Systems and Applications Conference.
- World Health Organization (2021) Falls. Available at: <https://www.who.int/news-room/fact-sheets/detail/falls> (Accessed: 3 September 2026).
- Smartphone Human Fall Dataset (2024) Train.csv and Test.csv, dataset archive supplied with the project evidence.
- Project Team (2026) Fall-model training outputs, source code, automated tests, simulation logs and integration screenshots. Unpublished project evidence.

Appendix A — Project Schedule

Week	Milestone	Evidence
1–2	Problem definition and requirements	Proposal and actor list
3–4	Dataset and sensor study	Data audit and wiring plan
5–7	Fall model and health rules	Training outputs and unit tests
8–9	Mood analysis and smoothing	Live preview and tests
10–11	Firebase and mobile integration	Payloads and app screens
12	System simulation and corrections	Passed test suite
13	Demonstration and documentation	Final demo and report

Table 22 Project Schedule

Appendix B — Test Summary

Test area	Representative cases	Result
Alert engine	Normal none; critical; fall-specific; SOS-specific	Pass
Emotion smoother	Continuous 4 s; low confidence; stale reading; long gap; slow detector	Pass
Health anomaly	Normal; warning; critical	Pass
System engine	Fall alert; mood result; normal health	Pass
Overall project state	Automated suite reported by project team	18 passed

Table 23 Appendix B — Test Summary

Appendix C — Reproduction Commands

Create environment and install dependencies: `python -m venv .venv;`
`.\venv\Scripts\python.exe -m pip install -r requirements.txt`

1. Run automated tests: `.\venv\Scripts\python.exe -m unittest discover -s tests -v`
2. Run health simulation: `.\venv\Scripts\python.exe simulate_health_monitor.py`

3. Run fall simulation: `.\venv\Scripts\python.exe simulate_fall_monitor.py`
4. Run mood detection: `.\venv\Scripts\python.exe run_mood_detection.py`

The proposal provides the project WBS and implementation timeline. Actual organization-issued letters, completed questionnaires/interview records, meeting minutes, log sheets, and supervisor-reviewed documents were not included in the proposal source and therefore should be attached by the project team before final submission.

Appendix D — Required Completion Items

1. Replace all candidate, index, supervisor, course-director, and date placeholders.
2. Confirm the exact course/declaration wording with the supervisor.
3. Insert final mobile application screenshots and captions.
4. Insert actual organization letter, questionnaires/interviews, meeting minutes, log sheet, and supervisor-reviewed documents.
5. Attach final Firebase rules/schema screenshots with sensitive credentials hidden.
6. Update the Word table of contents (Ctrl+A, F9) and verify page numbers before printing.
7. Obtain supervisor approval before navy-blue hard binding.

